

SwiftSpec: Ultra-Low Latency LLM Decoding by Scaling Asynchronous Speculative Decoding

Ziyi Zhang^{1,2}, Ziheng Jiang^{1†}, Chengquan Jiang¹, Menghan Yu¹, Size Zheng¹,
Haibin Lin¹, Henry Hoffmann², Xin Liu^{1,†}

¹ByteDance Seed, ²University of Chicago

[†]Corresponding authors

Abstract

Low-latency decoding for large language models (LLMs) is crucial for applications like chatbots and code assistants, yet generating long outputs remains slow in single-query settings. Prior work on speculative decoding (which combines a small draft model with a larger target model) and tensor parallelism has each accelerated decoding. However, conventional approaches fail to apply both simultaneously due to imbalanced compute requirements (between draft and target models), KV-cache inconsistencies, and communication overheads under small-batch tensor-parallelism. This paper introduces SwiftSpec, a system that targets ultra-low latency for LLM decoding. SwiftSpec redesigns the speculative decoding pipeline in an asynchronous and disaggregated manner, so that each component can be scaled flexibly and remove draft overhead from the critical path. To realize this design, SwiftSpec proposes parallel tree generation, tree-aware KV cache management, and fused, latency-optimized kernels to overcome the challenges listed above. Across 5 model families and 6 datasets, SwiftSpec achieves an average of $1.75\times$ speedup over state-of-the-art speculative decoding systems and, as a highlight, serves Llama3-70B at 348 tokens/s on 8 Nvidia Hopper GPUs, making it the fastest known system for low-latency LLM serving at this scale.

Date: June 16, 2025

Correspondence: Ziheng Jiang at ziheng.jiang@bytedance.com, Xin Liu at liuxin.ai@bytedance.com

1 Introduction

The remarkable capacity of large language models (LLMs) to learn from vast datasets has been instrumental in enabling the rapid proliferation of emerging applications across diverse domains, including chatbots [13], search [4, 37], and personalized recommendation systems [13, 39]. Real-time applications, such as interactive code assistants [6, 11, 15] and robotics [41, 46], impose stringent limits on models’ decoding latency. Recently, Chain-of-thought (CoT) [9, 35] is also increasingly adopted to improve reasoning quality. A single CoT inference can decode tens of thousands of tokens and take more than ten

minutes to complete. Reducing decoding latency is, therefore, critical.

In LLM-serving systems, there is an inherent trade-off between throughput and latency under the same compute resources. This work investigates how to achieve ultra-low latency decoding in single-request scenarios, where existing serving frameworks—designed to maximize throughput under service-level-objective (SLO) constraints—often fall short. For instance, a 4-bit quantized Llama-70B model running on 8 NVIDIA H800 GPUs can take approximately 30s to generate a response of about 3000 tokens when deployed with the popular LLM serving frameworks.

SwiftSpec: 通过扩展异步推测解码实现超低延迟 LLM 解码

张子怡^{1,2}、姜子恒^{1†}、蒋成全¹、于梦涵¹、郑尺寸¹、林海滨¹、亨利·霍夫曼²、刘欣^{1,†}

¹ByteDance Seed, ²芝加哥大学

[†]通讯作者

抽象的

大型语言模型 (LLM) 的低延迟解码对于聊天机器人和代码助手等应用至关重要，但在单查询设置中生成输出仍然很慢。先前关于推测解码（将小型草稿模型与较大目标模型相结合）和张量并行性的工作都加速了解码。然而，由于计算需求不平衡（草稿模型和目标模型之间）、KV 缓存不一致以及小批量张量并行下的通信开销，传统方法无法同时应用这两种方法。

本文介绍了 SwiftSpec，这是一个以 LLM 解码超低延迟为目标的系统。SwiftSpec 以异步和分解的方式重新设计了推测解码管道，以便每个组件都可以灵活扩展并消除关键路径的草稿开销。为了实现这一设计，SwiftSpec 提出了并行树生成、树感知 KV 缓存管理以及融合的延迟优化内核来克服上述挑战。在 5 个模型系列和 6 个数据集中，SwiftSpec 比最先进的推测解码系统平均加速 1.75 ×，并且作为一个亮点，它在 8 个 Nvidia Hopper GPU 上以 348 个令牌/秒的速度为 Llama3-70B 提供服务，使其成为这种规模的低延迟 LLM 服务的已知最快系统。

日期：2025 年 6 月 16 日

通讯作者：姜子恒, ziheng.jiang@bytedance.com, 刘鑫, liuxin.ai@bytedance.com

1 简介

大型语言模型 (LLM) 从海量数据集中学习的卓越能力有助于跨不同领域的新兴应用程序的快速扩散，包括聊天机器人[13]、搜索[4, 37]和个性化推荐系统[13, 39]。实时应用程序，例如交互式代码助手 [6, 11, 15] 和机器人 [41, 46]，对模型的解码延迟施加了严格的限制。最近，思想链 (CoT) [9, 35] 也越来越多地被用来提高推理质量。一次 CoT 推理可以解码数万个 token，需要十多个

分钟即可完成。因此，减少解码延迟至关重要。

在 LLM 服务系统中，相同计算资源下的吞吐量和延迟之间存在固有的权衡。这项工作研究了如何在单请求场景中实现超低延迟解码，在这种情况下，现有的服务框架（旨在服务级别目标 (SLO) 约束下最大化吞吐量）经常达不到要求。例如，当与流行的 LLM 服务框架一起部署时，在 8 个 NVIDIA H800 GPU 上运行的 4 位量化 LLama-70B 模型可能需要大约 30 秒才能生成大约 3000 个令牌的响应。

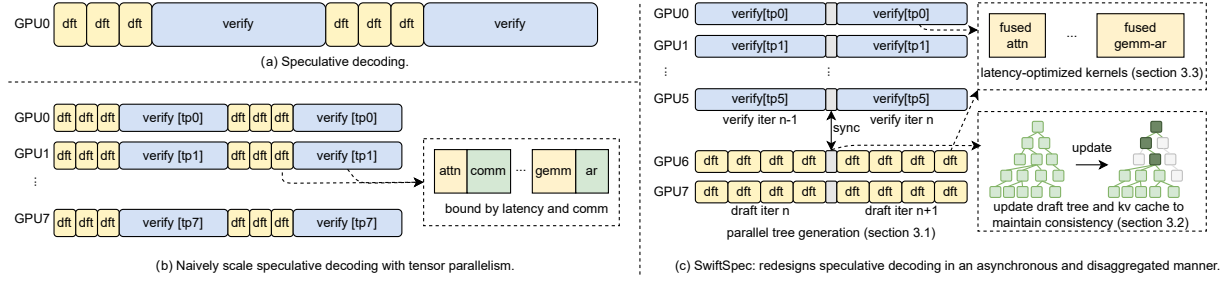


Figure 1 Overview of speculative decoding variants. (a) Conventional speculative decoding with sequential draft/verify steps. (b) Tensor-parallel decoding with reduced latency but communication overhead and GPU under utilization. (c) SwiftSpec: Our approach, combining parallel tree generation (§3.1), KV cache consistency (§3.2), and latency-optimized kernels (§3.3).

	# gpus=1	# gpus=2	# gpus=4	# gpus=8
Llama3-1B	1.58ms	1.58ms	1.73ms	1.72ms
Llama3-3B	2.77ms	2.61ms	2.80ms	3.27ms
Llama3-8B	4.30ms	3.46ms	3.50ms	3.78ms
Llama3-70B	24.78ms	15.90ms	11.86ms	11.22ms

Table 1 Time per inference for 4 different int4 AWQ models under Llama3 family under different degrees (ngpus) of tensor parallelism under a batch size of 8

Speculative decoding [3, 5, 18, 25] accelerates LLM inference in single-request scenarios. Speculative decoding consists of two distinct phases: a draft phase followed by a verification phase. During the draft phase, a relatively small draft model rapidly generates a sequence of candidate tokens (and, in some variants, a tree-structured set of candidates). During the subsequent verification phase, a significantly larger target model validates all candidates by performing a batch inference, thereby emitting multiple tokens at once and reducing decoding latency. Prior work typically treats the draft and verification phases as strictly sequential operations because of their data dependencies [3, 19, 25], as shown in Figure 1(a). This design places the draft phase on the critical path as an additional overhead, preventing speculative decoding from fully realizing its latency-reduction potential.

Tensor parallelism [30] is another technique to reduce the decoding latency by scaling the computation resources. Tensor parallelism partitions the model weights across multiple GPUs and then performs all-reduce operations to aggregate the partial results. However, a straightforward combination of tensor parallelism with speculative decoding, as shown in Figure 1(b), is ineffective. In speculative decoding, the draft and target models are co-located on the same devices. Because the two models differ greatly in size, applying the same degree of tensor-parallelism to both cannot yield optimal system latency. The smaller draft model

reaches the point of diminishing returns sooner: once its weights are already finely sharded, further increasing the tensor-parallelism no longer reduces latency, because other overheads—most notably inter-GPU communication—dominate. As Table 1 shows, under a batch size of 8, extending from 2 GPUs to 4 GPUs, a lightweight Llama 3B draft model has even longer inference time (increasing from 2.61ms to 2.80ms per inference), while the Llama 70B target model still has good improvements (decreasing from 15.90ms to 11.86ms).

To effectively combine speculative decoding with tensor parallelism and achieve ultra-low decoding latency, we redesign the speculative decoding process in an asynchronous, disaggregated manner (Figure 1(c)). We partition GPUs into two groups: verification and draft. Rather than co-located on the same hardware, the target model runs on the verification group, and the draft model runs on the draft group. The verification and draft phases proceed in parallel: while the verification group verifies iteration $n - 1$, the draft group concurrently produces candidates for iteration n . When a verification iteration is complete, the verification group synchronizes the validated tokens with the draft group and obtains the next set of candidate tokens to be verified. Under this design, (1) the draft and target models can be flexibly scaled to different degrees of parallelism, and (2) the dependencies between the two phases are decoupled, removing the draft phase from the critical path.

Realizing this design poses three system-level challenges. First, while the verification group is still performing parallel validation and has not yet obtained a definitive answer for the current iteration, the draft group must still generate the candidate set for the next iteration. Second, maintaining key-value cache consistency between complex drafting models

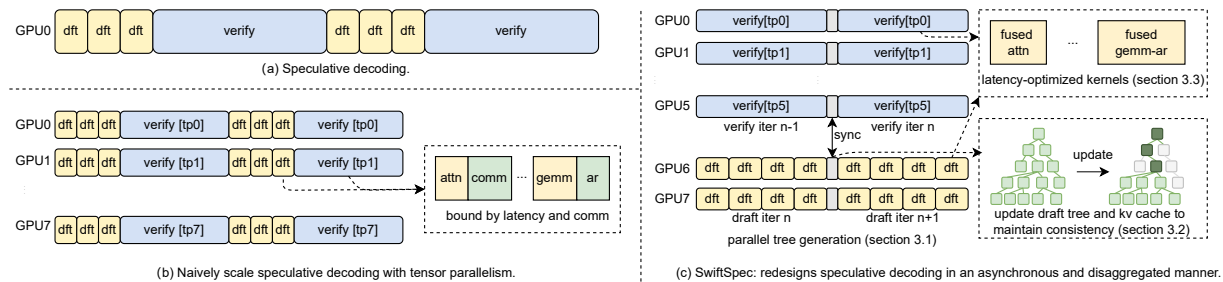


图 1 推测性解码变体概述。(a)具有顺序草稿/验证步骤的传统推测解码。(b)张量并行解码，延迟减少，但通信开销和 GPU 利用率较低。(c) SwiftSpec：我们的方法，结合了并行树生成（第 3.1 节）、KV 缓存一致性（第 3.2 节）和延迟优化内核（第 3.3 节）。

	# gpus=1	# gpus=2	# gpus=4	# gpus=8
Llama3-1B	1.58ms	1.58ms	1.73ms	1.72ms
Llama3-3B	2.77ms	2.61ms	2.80ms	3.27ms
Llama3-8B	4.30ms	3.46ms	3.50ms	3.78ms
Llama3-70B	24.78ms	15.90ms	11.86ms	11.22ms

表 1 在批量大小为 8 的情况下，Llama3 系列下的 4 个不同的 int4 AWQ 模型在不同张量并行度（ngpus）下的每次推理时间

推测性解码 [3,5,18,25] 可加速单请求场景中的 LLM 推理。推测解码由两个不同的阶段组成：草稿阶段和验证阶段。在草稿阶段，相对较小的草稿模型快速生成一系列候选标记（在某些变体中，生成树状结构的候选标记集）。在随后的验证阶段，一个更大的目标模型通过执行批量推理来验证所有候选者，从而一次发出多个令牌并减少解码延迟。先前的工作通常将草稿和验证阶段视为严格的顺序操作，因为它们具有数据依赖性[3,19,25]，如图 1(a) 所示。此设计将草稿阶段作为额外开销放置在关键路径上，从而阻止推测解码完全实现其减少延迟的潜力。

张量并行[30]是另一种通过扩展计算资源来减少解码延迟的技术。张量并行性将模型权重划分到多个 GPU 上，然后执行全归约操作来聚合部分结果。然而，如图 1(b) 所示，张量并行性与推测解码的直接组合是无效的。在推测解码中，草稿模型和目标模型位于同一设备上。由于这两个模型的大小差异很大，因此对两者应用相同程度的张量并行性无法产生最佳的系统延迟。较小的模型

更快达到收益递减点：一旦其权重已经被精细分片，进一步增加张量并行性就不再减少延迟，因为其他开销（尤其是 GPU 间通信）占主导地位。如表 1 所示，在批量大小从 2 个 GPU 扩展到 4 个 GPU 的情况下，轻量级 Llama 3B 草稿模型具有更长的推理时间（每次推理从 2.61ms 增加到 2.80ms），而 Llama 70B 目标模型仍然有很好的改进（从 15.90ms 减少到 11.86ms）。

为了有效地将推测解码与张量并行性结合起来并实现超低解码延迟，我们以异步、分解的方式重新设计了推测解码过程（图 1 (c)）。我们将 GPU 分为两组：验证组和草案组。目标模型不是在同一硬件上共存，而是在验证组上运行，草稿模型在草稿组上运行。验证和草案阶段并行进行：当验证组验证迭代 $n-1$ 时，草案组同时生成迭代 n 的候选者。当验证迭代完成时，验证组将已验证的令牌与草案组同步，并获取下一组要验证的候选令牌。在这种设计下，（1）草案和目标模型可以灵活地扩展到不同程度的并行性，（2）两个阶段之间的依赖关系被解耦，将草案阶段从关键路径中删除。

实现这一设计面临三个系统级挑战。首先，虽然验证组仍在进行并行验证并且尚未获得当前迭代的明确答案，但草案组仍必须生成下一次迭代的候选集。二、维护复杂绘图模型之间的键值缓存一致性

(e.g., tree-structured draft models) and the target model is non-trivial. When tree-based draft generation runs in parallel, newly accepted tokens may force the draft model to discard invalid branches. It is important (yet challenging) to keep a consistent view of the KV cache of accepted tokens and the draft tokens that might be useful in the future. Third, hiding communication latency during decoding is challenging. For example, when draft and target models are under tensor parallelism, it is hard to overlap the all-reduce operation with other operations since they usually remain on the critical path. Furthermore, the GPU kernels, usually optimized for higher throughput, have suboptimal performance under low batch sizes, spending most of the time on the latency of data movement and kernel launch.

We present *SwiftSpec*, a novel system that achieves ultra-low decoding latency for LLMs (Figure 1(c)), significantly reducing the decoding latency in single-request scenarios. To address the above challenges, SwiftSpec introduces: (a) **Parallel tree generation**. We allocate the draft and target models onto different sets of GPUs, eliminating inter-dependencies and allowing each model to generate tokens or verify them independently. While the target model verifies one batch, the draft model simultaneously produces future candidate tokens, ensuring high GPU utilization. This allows us to scale each model according to its own compute requirements. (b) **Consistent KV cache management**. After each verification step, we carefully reorganize the KV Cache of both the draft and target models to maintain consistency. For the draft model, we develop a scheme to keep the accepted and future tokens consistent with the draft tree, even when some guesses are incorrect and some part of the draft tree is invalidated. This approach also maximizes the reuse of the previously computed KV cache values. (c) **Latency-optimized kernels**. We develop latency-optimized kernels that minimize synchronization barriers and unnecessary data transfers, accelerating inference in low-batch scenarios. Using the Nvidia Collective Communication Library’s Low Latency (NCCL LL) protocol, we develop a fused GEMM with all-reduce and an attention operator without any explicit synchronization barriers. Furthermore, we fuse the multiple operations in the Switched Gated Linear Unit (SwiGLU) operator to decrease latency.

We evaluate SwiftSpec using five different model families and six different datasets. SwiftSpec consistently outperforms the baselines, achieving an average of $1.75\times$ decoding speed using 8 GPUs over the best

baseline. As a highlight, SwiftSpec serves Llama3-70B with an average decoding speed of 348 output tokens per second, currently the fastest solution we are aware of using Nvidia Hopper GPUs.

Our contributions are:

- We identify the scalability challenges of speculative decoding under tensor parallelism in existing LLM serving systems.
- We present SwiftSpec, which integrates techniques including parallel tree generation, consistent KV cache management, and latency-optimized kernels to redesign speculative decoding in an asynchronous, disaggregated manner.
- We conduct a comprehensive evaluation of SwiftSpec across five model families and six benchmark datasets, in which SwiftSpec consistently outperforms the baselines.

2 Background and System-level Challenges

2.1 Parallelism for LLM Decoding

LLM inference (e.g., [28, 33, 34]) is typically accelerated through model parallelism, which distributes computation across multiple GPUs. The two most common strategies are intra-operator parallelism (such as tensor parallelism, which splits matrix multiplications across GPUs) [30] and inter-operator parallelism (such as pipeline parallelism, which distributes layers across devices) [10, 14].

In single-request serving, tensor parallelism (TP) is generally preferred. By splitting large matrix operations across GPUs and aggregating results via all-reduce, TP reduces compute latency. Pipeline parallelism, while useful for high-throughput scenarios, often adds per-token latency due to sequential layer execution.

However, TP alone is insufficient to achieve low latency in speculative decoding. Draft models are typically small and gain relatively little from TP, while large target models benefit more from increased TP degrees. These asymmetric scaling needs pose challenges in resource allocation and utilization. SwiftSpec directly addresses these issues through parallel tree generation (§3.1) and latency-optimized kernels (§3.3).

(例如, 树形结构的草图模型) 并且目标模型并不简单。当基于树的草稿生成并行运行时, 新接受的令牌可能会迫使草稿模型丢弃无效分支。保持已接受令牌的 KV 缓存和将来可能有用的草稿令牌的一致视图非常重要 (但具有挑战性)。第三, 在解码过程中隐藏通信延迟具有挑战性。例如, 当草稿模型和目标模型处于张量并行状态时, 很难将全归约操作与其他操作重叠, 因为它们通常保留在关键路径上。此外, 通常针对更高吞吐量进行优化的 GPU 内核在低批量大小下性能不佳, 大部分时间都花在数据移动和内核启动的延迟上。

我们提出了 SwiftSpec, 这是一种为 LLM 实现超低解码延迟的新颖系统 (图 1(c)), 显著降低了单请求场景中的解码延迟。为了解决上述挑战, SwiftSpec 引入了: (a) 并行树生成。我们将草稿模型和目标模型分配到不同的 GPU 组上, 消除相互依赖性并允许每个模型独立生成令牌或验证它们。当目标模型验证一批时, 草稿模型同时生成未来的候选令牌, 确保高 GPU 利用率。这使我们能够根据每个模型自己的计算要求来扩展它。(b) 一致的 KV 缓存管理。在每个验证步骤之后, 我们都会仔细地重新组织草稿模型和目标模型的 KV 缓存以保持一致性。对于草稿模型, 我们开发了一个方案来保持已接受和未来的令牌与草稿树一致, 即使某些猜测不正确并且草稿树的某些部分无效。这种方法还最大限度地重用了先前计算的 KV 缓存值。(c) 延迟优化内核。我们开发了延迟优化的内核, 可以最大限度地减少同步障碍和不必要的数据传输, 从而加速低批量场景中的推理。使用 Nvidia 集体通信库的低延迟 (NCCL LL) 协议, 我们开发了一个融合 GEMM 的全归约和注意力算子, 没有任何显式同步障碍。此外, 我们融合了开关门控线性单元 (SwiGLU) 运算符中的多个操作以减少延迟。

我们使用五个不同的模型系列和六个不同的数据集来评估 SwiftSpec。SwiftSpec 始终优于基准, 使用 8 个 GPU 实现的平均解码速度比最佳解码速度快 1.75 ×

基准。作为一个亮点, SwiftSpec 为 Llama3-70B 提供服务, 平均解码速度为每秒 348 个输出令牌, 这是目前我们所知的使用 Nvidia Hopper GPU 的最快解决方案。

我们的贡献是:

- 我们确定了现有 LLM 服务系统中张量并行性下推测解码的可扩展性挑战。
- 我们提出了 SwiftSpec, 它集成了并行树生成、一致的 KV 缓存管理和延迟优化内核等技术, 以异步、分解的方式重新设计推测解码。
- 我们对五个模型系列和六个基准数据集对 SwiftSpec 进行了全面评估, 其中 SwiftSpec 始终优于基准。

2 背景和系统级挑战

2.1 LLM 解码的并行性

LLM 推理 (例如, [28,33,34]) 通常通过模型并行性来加速, 模型并行性将计算分布在多个 GPU 上。两种最常见的策略是运算符内并行 (例如张量并行, 跨 GPU 拆分矩阵乘法) [30] 和运算符间并行 (例如管道并行, 跨设备分布层) [10, 14]。

在单请求服务中, 通常首选张量并行性 (TP)。通过在 GPU 上拆分大型矩阵运算并通过 all-reduce 聚合结果, TP 减少了计算延迟。管道并行性虽然对于高吞吐量场景很有用, 但由于顺序层执行, 通常会增加每个令牌的延迟。

然而, 仅 TP 不足以在推测解码中实现低延迟。草稿模型通常较小, 从 TP 中获得的收益相对较小, 而大型目标模型则从增加的 TP 度中受益更多。这些不对称的扩展需求给资源分配和利用带来了挑战。SwiftSpec 通过并行树生成 (§3.1) 和延迟优化内核 (§3.3) 直接解决这些问题。

System	Spec Type	Spec Parallel	Consistent KV-Cache Management	Latency-Optimized Kernels
SpecInfer, EAGLE [19, 25]	Tree	No	easy to manage	standard kernels
PEARL [21]	Sequence	Yes	easy to manage	standard kernels
PipeInfer [2]	Tree	Yes	no fine-grained re-use of draft cache	standard kernels/CPU operations
AMUSD [24]	Sequence	Yes	easy to manage	standard kernels
SGLang, vLLM [17, 44]	Tree	No	easy to manage	limited optimization
SwiftSpec (Ours)	Tree	Yes	finer-grained reorganization, zero waste	fused latency-optimized ops

Table 2 The comparison between SwiftSpec and other existing baselines

2.2 GPU Constraints for Low Batch Size

Modern GPUs feature high-bandwidth memory (HBM) and fast interconnects (e.g., NVLink), and they are typically optimized for large-batch throughput workloads. In small-batch inference, common in interactive LLM serving, GPU performance suffers due to underutilized kernels and communication overhead.

In transformer layers executed with TP, three operators dominate runtime: GEMM, attention, and all-reduce. Under small batch sizes (≤ 8), these operators exhibit poor bandwidth and compute utilization due to short execution and frequent synchronization. Some targeted optimizations, such as low-bit quantization [20, 36], reduce communication costs but fall short of a holistic solution.

Similarly, state-of-the-art LLM serving frameworks have reduced communication costs for small sizes by using the Nvidia collective communication library all-reduce operator [17, 44]. Again, this falls short of a holistic solution. SwiftSpec instead introduces latency-optimized kernels (§3.3) that fuse GEMM and all-reduce operators to greatly reduce overhead and enable fine-grained communication.

2.3 Tree-based Speculative Decoding

2.3.1 Overview of Tree-based Approaches

Speculative decoding methods leverage a smaller draft model to accelerate inference from a larger target model [3, 5, 18, 25]. These techniques typically proceed in two stages: a drafting phase and a verification phase. In the drafting phase, the draft model iteratively proposes a short sequence or a partial tree [25] of potential next tokens. In the subsequent verification phase, the target model verifies these proposed tokens by performing a batched forward pass over the drafted tokens and then sampling from the results. The draft model then proceeds to predict the next set of tokens, and this process is repeated until the target model concludes generation.

Conventional speculative decoding approaches impose dependencies between the target model and

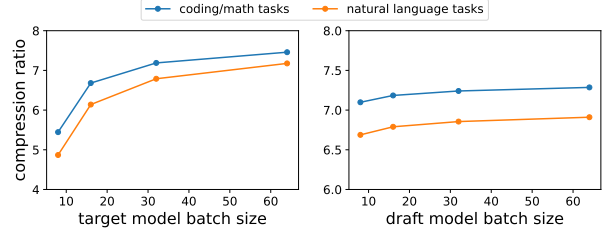


Figure 2 Compression ratio (average number of correctly guessed tokens per target model inference) while increasing the batch size of the target model (left figure) and draft model (right figure)

the draft model. Parallel sequence-based speculative decoding partially addresses this challenge by concurrently running the draft model to generate the next guesses (under the assumption that previous guesses are correct) while the target model verifies those preceding guesses [21, 24].

However, sequence-based methods typically exhibit lower-quality guesses compared to tree-based approaches. In tree-based speculative decoding, the draft model proposes a tree of candidate token paths instead of a linear sequence. The target model verifies a path through this tree. This approach tends to achieve higher compression ratios, i.e., tokens verified per target inference. Compression ratio directly impacts the end-to-end latency, as higher ratios reduce the number of target model invocations.

So, while tree-based methods can improve compression ratios, they also impose more complexity on the overall decoding system. First, while sequential approaches often generate the single most probable next token, it remains unclear how to optimally construct tree nodes in parallel to maximize the compression ratio. Second, managing the KV cache to maintain a consistent view of accepted tokens and potentially useful draft tree nodes introduces further complexity. SwiftSpec supports tree-based parallel speculative decoding by addressing these two limitations through two key innovations: parallel tree generation and consistency management in asynchronous decoding (§3.1–3.2).

System	Spec Type	Spec Parallel	Consistent KV-Cache Management	Latency-Optimized Kernels
SpecInfer, EAGLE [19, 25]	Tree	No	easy to manage	standard kernels
PEARL [21]	Sequence	Yes	easy to manage	standard kernels
PipeInfer [2]	Tree	Yes	no fine-grained re-use of draft cache	standard kernels/CPU operations
AMUSD [24]	Sequence	Yes	easy to manage	standard kernels
SGLang, vLLM [17, 44]	Tree	No	easy to manage	limited optimization
SwiftSpec (Ours)	Tree	Yes	fined-grained reorganization, zero waste	fused latency-optimized ops

表2 SwiftSpec与其他现有基线的比较

2.2 GPU 对低批量大小的限制

现代 GPU 具有高带宽内存 (HBM) 和快速互连 (例如 NVLink)，并且通常针对大批量吞吐量工作负载进行优化。在交互式 LLM 服务中常见的小批量推理中，GPU 性能由于未充分利用的内核和通信开销而受到影响。

在使用 TP 执行的 Transformer 层中，三个运算符在运行时占据主导地位：GEMM、attention 和 all-reduce。在小批量大小 (≤ 8) 下，由于执行时间短和同步频繁，这些运算符表现出较差的带宽和计算利用率。一些有针对性的优化，例如低位量化[20, 36]，可以降低通信成本，但达不到整体解决方案。

同样，最先进的 LLM 服务框架通过使用 Nvidia 集体通信库 all-reduce 运算符降低了小规模通信成本 [17, 44]。同样，这也不是一个整体的解决方案。相反，SwiftSpec 引入了延迟优化内核 (第 3.3 节)，它融合了 GEMM 和 all-reduce 运算符，以大大减少开销并实现细粒度通信。

2.3 基于树的推测解码

2.3.1 基于树的方法概述

推测性解码方法利用较小的草稿模型来加速较大目标模型的推理 [3,5,18,25]。这些技术通常分两个阶段进行：起草阶段和验证阶段。在起草阶段，草案模型迭代地提出潜在下一个标记的短序列或部分树[25]。在随后的验证阶段，目标模型通过对起草的令牌执行批量前向传递，然后从结果中采样来验证这些提议的令牌。然后，草稿模型继续预测下一组令牌，并重复此过程，直到目标模型完成生成。

传统的推测性解码方法在目标模型和模型之间强加了依赖性。

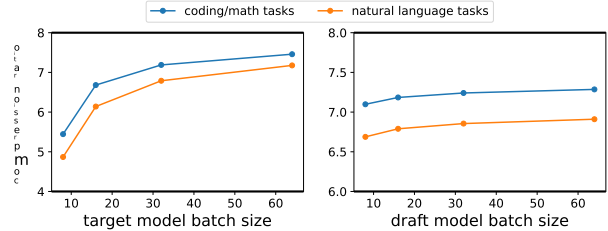


图2 增加目标模型（左图）和草稿模型（右图）批量大小时的压缩率（每个目标模型推理正确猜测的标记的平均数量）

草稿模型。基于并行序列的推测解码通过同时运行草稿模型来生成下一个猜测（假设先前的猜测是正确的），同时目标模型验证那些先前的猜测，从而部分解决了这一挑战[21, 24]。

然而，与基于树的方法相比，基于序列的方法通常表现出较低质量的猜测。在基于树的推测解码中，草案模型提出了候选标记路径树而不是线性序列。目标模型验证通过这棵树的路径。这种方法往往会实现更高的压缩比，即根据目标推理验证令牌。压缩比直接影响端到端延迟，因为较高的压缩比会减少目标模型调用的数量。

因此，虽然基于树的方法可以提高压缩比，但它们也给整个解码系统带来了更多的复杂性。首先，虽然顺序方法通常会生成单个最可能的下一个标记，但仍不清楚如何最佳地并行构造树节点以最大化压缩比。其次，管理 KV 缓存以维护已接受令牌和潜在有用的草稿树节点的一致视图会带来进一步的复杂性。SwiftSpec 通过两项关键创新解决这两个限制，支持基于树的并行推测解码：并行树生成和异步解码中的一致性管理（第 3.1–3.2 节）。

2.3.2 Low Batch Sizes in Tree-based Speculative Decoding

Figure 2 shows the effects of increasing the batch size for the target and draft models when serving a single request. We use Llama3 70B as the target model and Llama3 3B as the draft model, and we generate 7 tree layers (i.e., running the draft model 7 times) before running a batch (size 7) inference using the target model. In the first experiment (the left figure), we fix the batch size of the draft model to be 16, and the average number of guesses only increases marginally after the batch size is larger than 16. In the second experiment (the right figure), we fix the batch size of the target model to be 32, and when we increase the batch size of the draft model over 8, the average number of correct guesses of the target model almost stops increasing.

As we can see, to minimize the latency of serving a single request, it is usually sufficient to use a small batch size (≤ 16) for both the target and draft models. Note that this characteristic is different from throughput-oriented systems [1, 40, 45], where larger batch sizes improve system throughput while satisfying SLOs, rather than minimizing the end-to-end generation of a single request, which is the focus of our work.

2.4 Key Limitations of Prior Work

This section describes how existing speculative decoding systems fall short in addressing the following key system challenges in single-request serving: (1) independent scalability across draft and target models, (2) KV cache consistency under speculative execution, and (3) kernel under utilization under low batch sizes.

Table 2 summarizes the differences between recent work, and the following paragraphs provide a detailed analysis on why they fail to address those three challenges.

Lack of scalability of the target and draft models under low batch size. For sequential speculative decoding approaches, since there is a strict dependency between the target and draft models, tree-based speculative decoding can only run one draft model and the target model at the same time. Therefore, to use parallelism for speculative decoding, we have to scale both the draft and target models across all GPUs; otherwise, some GPUs will be idle when running the models. However, using more GPUs for TP does not necessarily decrease latency. Table 1 shows that the benefit of using more GPUs diminishes when we in-

crease the degree of tensor parallelism for the Llama model family. It also shows that different models could have different compute requirements. For example, the 70B model benefits from using 4 GPUs instead of 2, while there is no benefit from using more than 2 GPUs for the smaller models (1B, 3B, 8B). Prior work using sequential speculative decoding (SpecInfer[25], EAGLE [19], etc) fails to address this lack of scalability, while our design of parallel tree generation does (§3.1).

Lack of effective KV Cache management scheme for parallel tree-based speculative decoding. Tree-based speculative decoding imposes a challenge on consistent and effective KV cache management. While parallel sequence-based speculative decoding (PEARL[21], AMUSD[24]) has a concise way to manage the KV cache, only generating a chain of draft tokens yields a suboptimal compression ratio and thus low end-to-end performance. It is non-trivial to keep the consistency between the KV cache of both models and the draft trees. PipeInfer [2] runs draft tree generation and target model verification in parallel and generates multiple draft trees to fill up the bubbles during execution. However, if some draft trees get invalidated, every draft tree generated after that will be discarded, wasting the compute and yielding low effective utilization. Therefore, to achieve a high performance, it is important—but challenging—to keep the KV cache consistent while maximizing the reuse of generated tokens. Our consistency management scheme addresses this (§3.2).

Low utilization of the GPU kernels. While the computation and communication kernels are usually well-optimized for large batch inference, with large computation and communication demands, they do not perform well under low batch size.

Table 3 shows the amount of bandwidth and compute utilization of the most time-consuming kernels in a transformer layer of a Llama 70B model split across 4 GPUs under a batch size of 8. The bandwidth utilization shown for the all-reduce operation is the NVLink bandwidth, while for the other operators, it is the HBM to SRAM bandwidth utilization. Under this low batch size, all operators are communication-intensive, yet the bandwidth utilization (both across GPUs and within a GPU) is low ($< 10\%$) for both the all-reduce and attention operators. This is because the amount of communication is small, and therefore, the time is mainly spent on synchronization and waiting for the first input to arrive in the GPU SRAM. Some frameworks (vLLM [17], SGLang [44], etc) use the NCCL LL protocol to reduce the latency in the

2.3.2 基于树的推测解码中的低批量大小

图 2 显示了在服务单个请求时增加目标模型和草稿模型的批量大小的效果。我们使用 Llama3 70B 作为目标模型，Llama3 3B 作为草稿模型，并在目标模型运行批量（大小 7）推理之前生成 7 个树层（即运行草稿模型 7 次）。在第一个实验（左图）中，我们将草稿模型的batch size固定为16，当batch size 大于16后，平均猜测次数仅略有增加。在第二个实验（右图）中，我们将目标模型的batch size固定为32，当我们草稿模型的batch size增加到超过8时，目标模型的平均猜测正确次数几乎停止增加。

正如我们所看到的，为了最大限度地减少服务单个请求的延迟，通常对目标模型和草稿模型使用小批量大小（ ≤ 16 ）就足够了。请注意，此特征不同于面向吞吐量的系统[1,40,45]，较大的批量大小可以在满足SLO的同时提高系统吞吐量，而不是最小化单个请求的端到端生成，这是我们工作的重点。

2.4 先前工作的主要局限性

本节描述现有的推测性解码系统在解决单请求服务中的以下关键系统挑战方面有何不足：（1）跨草案和目标模型的独立可扩展性，（2）推测执行下的KV缓存一致性，以及（3）低批量大小下的内核利用率。

表 2 总结了最近工作之间的差异，以下段落详细分析了它们为何未能解决这三个挑战。

低批量下目标模型和草稿模型缺乏可扩展性。对于顺序推测解码方法，由于目标模型和草稿模型之间存在严格的依赖性，因此基于树的推测解码只能同时运行一个草稿模型和目标模型。因此，要使用并行性进行推测性解码，我们必须在所有 GPU 上扩展草稿模型和目标模型；否则，运行模型时某些 GPU 将处于空闲状态。然而，使用更多 GPU 进行 TP 并不一定会降低延迟。表 1 显示，当我们使用更多 GPU 时，使用更多 GPU 的好处就会减弱：

增加 Llama 模型系列的张量并行度。它还表明不同的模型可能有不同的计算要求。例如，70B 型号受益于使用 4 个 GPU（而不是 2 个），而对于较小型号（1B、3B、8B）使用 2 个以上 GPU 则没有任何好处。先前使用顺序推测解码的工作（SpecInfer[25]、EAGLE[19]等）未能解决这种可扩展性的缺乏，而我们的并行树生成设计却可以（§3.1）。

缺乏有效的基于树的并行推测解码的 KV 缓存管理方案。基于树的推测解码对一致且有效的 KV 缓存管理提出了挑战。虽然基于并行序列的推测解码（PEARL [21]，AMUSD [24]）有一种简洁的方式来管理 KV 缓存，但仅生成草稿令牌链会产生次优的压缩比，从而导致端到端性能较低。保持两个模型的 KV 缓存和草稿树之间的一致性并非易事。PipeInfer [2]并行运行草稿树生成和目标模型验证，并生成多个草稿树以在执行过程中填充气泡。但是，如果某些草稿树失效，则此后生成的所有草稿树都将被丢弃，从而浪费计算量并产生低有效利用率。因此，为了实现高性能，保持 KV 缓存的一致性非常重要，同时也具有挑战性，同时最大限度地重用生成的令牌。我们的一致性管理方案解决了这个问题（§3.2）。

GPU 内核利用率低。虽然计算和通信内核通常针对大批量推理进行了很好的优化，但在计算和通信需求较大的情况下，它们在低批量大小下表现不佳。

表 3 显示了 LLama 70B 模型的转换器层中最耗时的内核的带宽量和计算利用率，该模型在批量大小为 8 的情况下分布在 4 个 GPU 上。all-reduce 操作显示的带宽利用率是 NVLink 带宽，而对于其他运算符，它是 HBM 到 SRAM 的带宽利用率。在这种低批量大小下，所有算子都是通信密集型的，但 all-reduce 算子和注意力算子的带宽利用率（跨 GPU 和 GPU 内）都很低（ $< 10\%$ ）。这是因为通信量较小，因此时间主要花在同步和等待第一个输入到达 GPU SRAM 上。一些框架（vLLM [17]、SGLang [44]等）使用 NCCL LL 协议来减少延迟

Operators	Time (in us)	Comp. Util. (%)	Band. Util. (%)
QKV projection	16.9	2.0%	18.7%
mask attention	18.8	<0.01%	6.5%
O projection	10.8	2.5%	23.3%
all reduce	12.0	<0.01%	8.5%
SwiGLU	39.3	4.8%	44.6%
down projection	18.1	5.2%	48.5%
all reduce	15.3	<0.01%	6.6%

Table 3 The utilization of the most time-consuming kernels in an int4 AWQ Llama 70B model when running TP on 4 GPUs.

all reduce operator when communication volume is low. However, as shown in Table 2, there is still a significant opportunity to improve kernels to reduce latency for small batch sizes. Our latency-optimized kernels (in §3.3) fuse multiple optimizations to realize this opportunity.

3 SwiftSpec System Design and Architecture

SwiftSpec addresses the three key systems challenges identified in the prior section through a modular design built around: (1) parallel tree generation, which enables asynchronous decoding and independent GPU allocation (§3.1); (2) KV-cache consistency management, which supports reuse and correctness under speculative execution (§3.2); and (3) latency-optimized fused kernels, which reduce communication and compute overhead under tensor parallelism (§3.3). We describe each component in turn.

3.1 Parallel Tree Generation

To enable independent scaling of draft and target models, we introduce parallel tree generation, which splits the decoding process across GPU groups separately dedicated to drafting and verification. This allows both models to operate concurrently and avoids placing the draft phase on the critical path. The two groups communicate using NVLink/cross-network interconnect. The draft tree GPUs manage the draft tree and run draft inference to generate new tree nodes, while the target GPUs run the target model. Both the draft and target models are split across their respective GPUs through tensor parallelism (TP). GPUs computing the same model are connected tightly using NVLink.

Algorithm 1 details the interaction between draft and target models in each decoding iteration. Denote one round (or one iteration) as the procedures of the draft and target model between two synchronization points. Define bs as the batch size of the target model, w as the number of leaves for which we run the draft

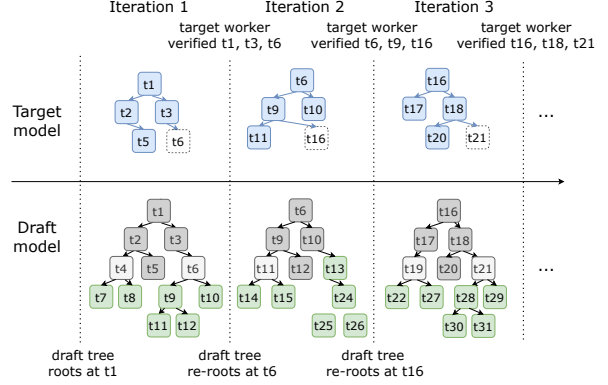


Figure 3 Example of parallel tree generation. Each iteration re-roots the tree based on verified tokens, while the KV-cache is reorganized to preserve useful state.

model inference (and thus expand the leaves to get potential children) each round (i.e., the batch size of the draft model), d as the number of tree expansions in one round. Both target worker and draft worker run in a loop until the end of the generation, and synchronize when each finishes one iteration in the loop.

In one iteration, the draft worker expands the draft tree d times, by running inference on w unexpanded leaves (i.e., the leaves where the KV-cache and logits (probability distribution of the next token) are not yet calculated) from the tree with the highest probability. After that, it synchronizes with the target worker to get the verified tokens. Then it re-roots the draft tree by walking down the tree using the path representing the verified tokens and adjusts the KV cache to stay consistent. After that, it grows the draft tree if there are not enough nodes to send to the target, and then it sends a sub-graph of the draft tree of size bs to the target worker.

On the other hand, in one iteration, the target model constantly gets the draft tokens from the draft tree and runs batch inferences to calculate the logits. After that, it samples through the logits to generate the tokens one by one and then sends the verified tokens back to the draft worker.

Figure 3 shows three decoding iterations. In each iteration, the draft model grows the tree while the target model verifies a subgraph. The tree is then re-rooted, and verified tokens are promoted to the KV cache. In this example, $bs = 4$, $d = 3$, $w = 2$.

At the start, the draft tree is $t_1, t_2, t_3, t_4, t_5, t_6$, and the draft workers select the top $bs = 4$ tokens (t_1, t_2, t_3, t_5) to give as $input_1$ to the target work-

Operators	Time (in us)	Comp. Util. (%)	Band. Util. (%)
QKV projection	16.9	2.0%	18.7%
mask attention	18.8	<0.01%	6.5%
O projection	10.8	2.5%	23.3%
all reduce	12.0	<0.01%	8.5%
SwiGLU	39.3	4.8%	44.6%
down projection	18.1	5.2%	48.5%
all reduce	15.3	<0.01%	6.6%

表 3 在 4 个 GPU 上运行 TP 时，int4 AWQ Llama 70B 模型中最耗时的内核的利用率。

当通信量较低时，所有操作员都会减少。然而，如表 2 所示，仍然有很大的机会改进内核以减少小批量的延迟。我们的延迟优化内核（第 3.3 节）融合了多种优化来实现这一机会。

3 SwiftSpec 系统设计与架构

SwiftSpec 通过围绕以下内容构建的模块化设计解决了上一节中确定的三个关键系统挑战：（1）并行树生成，它支持异步解码和独立 GPU 分配（第 3.1 节）；（2）KV-cache 一致性管理，支持推测执行下的重用和正确性（§3.2）；（3）延迟优化的融合内核，可减少张量并行下的通信和计算开销（第 3.3 节）。我们依次描述每个组件。

3.1 并行树生成

为了实现草稿模型和目标模型的独立扩展，我们引入了并行树生成，它将解码过程划分为分别专用于草稿和验证的 GPU 组。这允许两个模型同时运行，并避免将草稿阶段置于关键路径上。两组使用 NVLink/跨网络互连进行通信。草图树 GPU 管理草图树并运行草图推理以生成新的树节点，而目标 GPU 运行目标模型。草稿模型和目标模型都通过张量并行性 (TP) 划分到各自的 GPU 上。计算相同模型的 GPU 使用 NVLink 紧密连接。

算法 1 详细介绍了每次解码迭代中草稿模型和目标模型之间的交互。将一轮（或一次迭代）表示为两个同步点之间的草稿和目标模型的过程。将 bs 定义为目标模型的批量大小，将 w 定义为我们运行草稿的叶数

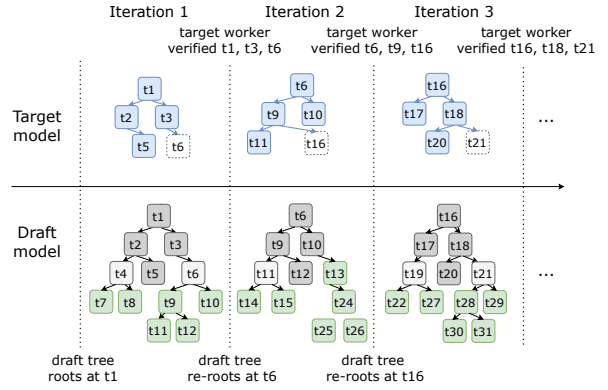


图 3 并行树生成示例。每次迭代都会根据已验证的令牌重新确定树的根，同时重新组织 KV 缓存以保留有用的状态。

每轮模型推理（从而扩展叶子以获得潜在的子节点）（即草稿模型的批量大小）， d 作为一轮中树扩展的数量。目标工作程序和草稿工作程序都在循环中运行，直到生成结束，并在各自完成循环中的一次迭代时进行同步。

在一次迭代中，草稿工作人员通过对具有最高概率的树中的 w 个未展开的叶子（即尚未计算 KV 缓存和 logits（下一个令牌的概率分布）的叶子）运行推理，将草稿树展开 d 次。之后，它与目标worker同步以获取经过验证的令牌。然后，它通过使用表示已验证令牌的路径沿着树向下走来重新确定草稿树的根，并调整 KV 缓存保持一致。之后，如果没有足够的节点发送到目标，它会增长草稿树，然后将大小为 bs 的草稿树的子图发送到目标工作程序。

另一方面，在一次迭代中，目标模型不断从草稿树中获取草稿标记并运行批量推理来计算 logits。之后，它对 logits 进行采样，一一生成令牌，然后将验证后的令牌发送回草稿工作人员。

图 3 显示了三个解码迭代。在每次迭代中，草稿模型都会生长树，而目标模型会验证子图。然后树会重新生根，并且经过验证的令牌会被提升到 KV 缓存。在此示例中， $bs = 4$ 、 $d = 3$ 、 $w = 2$ 。

一开始，草稿树是 $t_1, t_2, t_3, t_4, t_5, t_6$ ，草稿工人选择顶部的 $bs = 4$ 个令牌 (t_1, t_2, t_3, t_5) 作为 $input_1$ 提供给目标工作 -

ers. During iteration 1, while the draft workers continue growing the tree with 6 new nodes, the target workers run inference on $input_1$ and sample $output_1 = (t_1, t_3, t_6)$. Then, the draft workers verify that (t_1, t_3, t_6) is a valid path in the tree and re-root at t_6 . With enough nodes remaining, they choose the next top 4 tokens $(t_6, t_9, t_{10}, t_{11})$ as $input_2$. During iteration 2, the draft workers grow 6 more nodes while the target workers process $input_2$ and produce $output_2 = (t_6, t_9, t_{16})$. However, t_{16} is not yet in the tree, so the draft workers re-root at t_{16} and keep growing new nodes $t_{17}, t_{18}, t_{19}, t_{20}, t_{21}$, giving $(t_{16}, t_{17}, t_{18}, t_{20})$ as $input_3$. During iteration 3, a similar process continues, with the draft and target workers running in parallel, growing and verifying tokens as they build out the tree.

Maximum-likelihood tree expansion We use the logarithm of the softmax probability as the value of each node, and use the sum of values from the root to each node as the weight. Thus, a higher weight means a higher probability that a token could be generated (under the distribution of the draft model). We keep the pair $(value, node)$ in a priority queue to efficiently get the most probable leaves in $O(k \log s)$, where s is the number of probable leaves to consider and expand the tree.

GPU allocation for draft model and target model

Given a GPU node of k GPUs, we will allocate x ($1 \leq x \leq k-1$) GPUs to the target model and $(k-x)$ GPUs to the draft model. To determine which x to use, we run a profile phase before serving the queries, where we try out different x s to find which configuration yields the fastest average decoding speed. We found that if we fix the target model, the optimal x is smaller when we are using a more powerful draft model (more analysis is in 5.5)

Setting the batch size Larger bs, w will lead to higher acceptance ratio per iteration, but when bs, w get larger and larger, the margin gain on the acceptance ratio will decrease, and total running time will increase. We set $bs = 8$ and $w = 8$ empirically to balance the acceptance ratio and running time, based on our analysis in 5.5.

Setting the number of tree expansions d in one round.

Before we start serving the requests, we first profile the running time of both the draft model and the target model. Denote t_{target} as one round of target model inference, and t_{draft} as one round of draft tree expansion. Define $r = \lfloor \frac{t_{target}}{t_{draft}} \rfloor$. We set $d = r$ or $d = r + 1$ so that draft tree expansion and the target model verification finish nearly at the same time to

Input: Depth of tree to generate d , width of tree to generate w , batch size of target model bs

```

1 if Is_draft_worker then
2   while True do
3     for  $i = 1$  to  $d$  do
4       Expand the  $w$  most probable leaves from
         the tree;
5       Get verified tokens from the target worker;
6       if target worker signal to stop then
7         break;
8       Update KV Cache and draft tree based on
         verified tokens;
9       while Tree size  $< bs$  do
10        Expand the  $w$  most probable leaves from
           the tree;
11        Get the most probable subgraph of size  $bs$ 
           from the draft tree;
12        Send it to the target worker to verify;
13 else if Is_target_worker then
14   while True do
15     Get draft tokens from the draft tree worker;
16     Verify the draft tokens by batch inference;
17     if Reach the end of generation then
18       Send the stop signal to the draft worker;
19       Break;
20   else
21     Send the verified tokens to the draft tree
       worker;
```

Algorithm 1: Parallel tree generation algorithm

maximize parallelism.

Non-square mask support for efficient masked attention kernel

The attention operator in the target model uses a square mask, since the target model takes a tree each time, and each token will only mask out the attention with those tokens that are not the ancestors within the current input. This is similar to prior work [25]. However, for the draft model, this is not the case. Consider the example in Figure 4 with a current tree of size 6, and we want to calculate the logits of 4 probable leaves, then regarding the tree cache, we only calculate the attention of each leave with its ancestor on the tree (and also all the data that is in the prefix cache). In this case, we need a mask of at least size $(4, 10)$ to contain all the necessary information. Therefore, we support a non-square mask as input in our attention operator for the draft model.

3.2 KV Cache Consistency Management

To maintain the consistency between the draft model and target model, we develop a consistency management scheme to reorganize the KV cache of the draft model so that it remains consistent with the target

呃。在第 1 次迭代期间，当草稿工作人员继续使用 6 个新节点生长树时，目标工作人员对 $input_1$ 和样本 $output_1 = (t_1, t_3, t_6)$ 运行推理。然后，草稿工人验证 (t_1, t_3, t_6) 是树中的有效路径并在 t_6 处重新扎根。在剩余足够的节点的情况下，他们选择下一个前 4 个令牌 $(t_6, t_9, t_{10}, t_{11})$ 作为 $input_2$ 。在第 2 次迭代期间，草稿工作线程又增长了 6 个节点，而目标工作线程处理 $input_2$ 并生成 $output_2 = (t_6, t_9, t_{16})$ 。然而， t_{16} 尚未在树中，因此征兵人员在 t_{16} 处重新扎根并继续生长新节点 $t_{17}, t_{18}, t_{19}, t_{20}, t_{21}$ ，将 $(t_{16}, t_{17}, t_{18}, t_{20})$ 指定为 $input_3$ 。在第 3 次迭代期间，类似的过程继续进行，草稿和目标工作人员并行运行，在构建树时增长和验证令牌。

最大似然树扩展 我们使用 softmax 概率的对数作为每个节点的值，并使用从根到每个节点的值之和作为权重。因此，较高的权重意味着生成代币的概率较高（在草稿模型的分布下）。我们将这对 $(value, node)$ 保留在优先级队列中，以有效地获取 $O(k \log s)$ 中最可能的叶子，其中 s 是要考虑和扩展树的可能叶子的数量。

草稿模型和目标模型的 GPU 分配 给定一个包含 k 个 GPU 的 GPU 节点，我们将为目标模型分配 $x (1 \leq x \leq k-1)$ 个 GPU，为草稿模型分配 $(k-x)$ 个 GPU。为了确定使用哪个 x ，我们在提供查询之前运行配置文件阶段，在此阶段我们尝试不同的 x 来找出哪种配置产生最快的平均解码速度。我们发现，如果我们修复目标模型，当我们使用更强大的草图模型时，最优的 x 会更小（更多分析在 5.5 中）

设置批量大小较大的 bs, w 将导致每次迭代的接受率更高，但是当 bs, w 变得越来越大时，接受率的裕度增益将减小，并且总运行时间将增加。根据我们在 § 5.5 中的分析，我们凭经验设置 $bs = 8$ 和 $w = 8$ 来平衡接受率和运行时间。

设置一轮中树扩展 d 的数量。在开始服务请求之前，我们首先分析草稿模型和目标模型的运行时间。将 t_{target} 表示为一轮目标模型推理，将 t_{draft} 表示为一轮草稿树扩展。定义 $r = \lfloor \frac{t_{target}}{t_{draft}} \rfloor$ 。我们将 $d = r$ 或 $d = r + 1$ 设置为 1，以便草稿树扩展和目标模型验证几乎同时完成

输入：生成 d 的树的深度，要生成的树的宽度
generate w , batch size of target model bs

```

1 if Is_draft_worker then
2   while True do
3     for  $i = 1$  to  $d$  do
4       Expand the  $w$  most probable leaves from
         the tree;
5       Get verified tokens from the target worker;
6       if target worker signal to stop then
7         break;
8       Update KV Cache and draft tree based on
         verified tokens;
9       while Tree size  $< bs$  do
10        Expand the  $w$  most probable leaves from
           the tree;
11        Get the most probable subgraph of size  $bs$ 
           from the draft tree;
12        Send it to the target worker to verify;
13 else if Is_target_worker then
14   while True do
15     Get draft tokens from the draft tree worker;
16     Verify the draft tokens by batch inference;
17     if Reach the end of generation then
18       Send the stop signal to the draft worker;
19       Break;
20   else
21     Send the verified tokens to the draft tree
       worker;
```

Algorithm 1: Parallel tree generation algorithm

最大化并行性。

非方形掩码支持有效的掩码注意内核目标模型中的注意算子使用方形掩码，因为目标模型每次都采用一棵树，并且每个标记只会用那些不是当前输入中祖先的标记来掩盖注意。这与之前的工作类似[25]。然而，对于草案模型来说，情况并非如此。考虑图 4 中的示例，当前树大小为 6，我们想要计算 4 个可能叶子的 logits，那么对于树缓存，我们只计算每个叶子及其祖先在树上的注意力（以及前缀缓存中的所有数据）。在这种情况下，我们需要一个至少大小为 (4, 10) 的掩码来包含所有必要的信息。因此，我们支持非方形掩码作为草稿模型的注意力运算符的输入。

3.2 KV缓存一致性管理

为了保持草稿模型和目标模型之间的一致性，我们开发了一种一致性管理方案来重新组织草稿模型的 KV 缓存，使其与目标模型保持一致

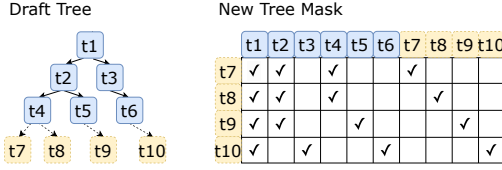


Figure 4 Example of a non-square tree mask during draft tree expansion: yellow nodes are the leaves to expand, and the blue nodes are the existing tree nodes

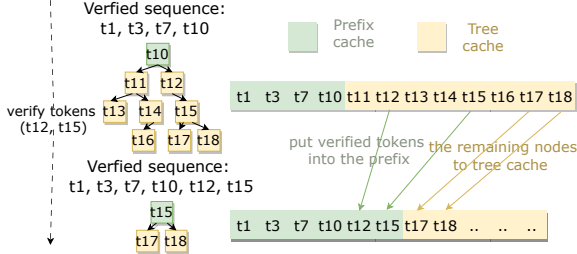


Figure 5 KV cache management of the draft model: each time after the new verified tokens get updated, the KV states of the verified tokens will be in the prefix of the KV cache (green part), and the KV states of the draft tree tokens will be right after the prefix (yellow part).

model and the draft tree and maximizes the re-use of previously computed KV states.

Throughout the execution, we organize the KV Cache of the draft model as follows and keep it as an invariant throughout the execution: the KV states of the verified tokens are stored continuously in the prefix of the KV cache (which we call prefix cache), and the KV states of the tree are stored right after the prefix (which we call tree cache).

While the target worker is doing batch inference and sampling on the tokens from the draft workers, the draft worker keeps generating new tree leaves in the draft tree, appending the tree cache after the existing entries.

Re-organization of KV cache for verified tokens After the target worker samples the tokens, it sends the verified tokens to the draft worker. The draft worker then walks through the tree using the verified tokens and reroots at the last verified token. Then, if the last verified token exists in the current draft tree, then we will reorganize the tree cache so that only the KV states of the nodes in the new subtree remain in the tree cache. In this way, even when some of the predicted tokens we send to the target worker are wrong, we can still reuse all the computed KV states in the subtree, avoiding any recomputation.

Figure 5 shows an example of how the KV cache of the draft model is updated when there are new verified tokens. Suppose the sequence (t_1, t_3, t_7, t_{10}) is already verified, and the prefix of the cache is the KV states of those tokens, and the KV states of the draft tree tokens are organized contiguously after the prefix. When we update the verified tokens to be $(t_1, t_3, t_7, t_{10}, t_{12}, t_{15})$, we walk down the draft tree using the newly verified tokens (t_{12}, t_{15}) . Then we reach the node t_{15} , which means the nodes in the subtree, t_{17}, t_{18} , are still useful in the future. Therefore, we move t_{12}, t_{15} to the prefix cache so that it stores the information of the same verified tokens as the target model. Then it reorganizes the remaining sub-tree of t_{15} (i.e. t_{17}, t_{18}) into the next positions available, discarding the KV states that are no longer useful (e.g. t_{11}, t_{12} , etc).

In the case where the draft tree does not have enough nodes to send back to the target worker, it expands bs nodes immediately using one draft model inference. In either case, the draft tree will have enough nodes to pass to target workers, therefore entering the next iteration, with the KV states synchronized across the draft model, the target model, and the draft tree.

3.3 Latency-Optimized Kernels Design

To reduce the inference time of both draft and target under low batch size, we design and implement *latency-optimized* operators for the Llama and Qwen model family. While our design could be applied to any precision, we implement it for the int4 AWQ quantized model. The operators we optimize include all-reduce, masked attention, and SwiGLU. We first introduce the Nvidia Collective Communication Library’s Low Latency (NCCL LL) protocol, which our work leverages heavily.

NCCL LL protocol. This is a communication scheme to reduce the latency of GPU-to-GPU send and receive operations. Algorithm 2 shows the store and load function of the LL protocol. The functions *AtomicStore* and *AtomicLoad* are NVidia GPU PTX instructions *ld.volatile.global.v4.u32* and *st.volatile.global.v4.u32*, which load and store 16 bytes of data from and to the global memory in the same or different GPUs. Each 8-byte (2 32-bit unsigned integers) load/store is atomic.

The storeLL function takes a 64-bit integer *val* and a 32-bit integer flag, splits *val* into two 32-bit integers, and stores them each with *flag* to a memory location. The loadLL function takes a memory location and a flag. It keeps polling until the flag at the memory lo-

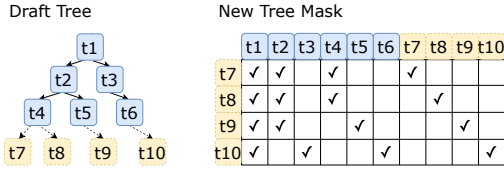


图 4 草稿树扩展期间的非方形树掩码示例：黄色节点是要扩展的叶子，蓝色节点是现有的树节点

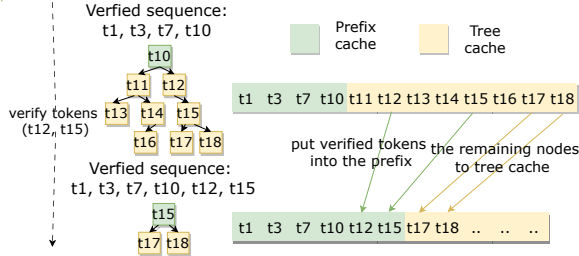


图5 草稿模型的KV缓存管理：每次新的验证令牌更新后，验证令牌的KV状态将位于KV缓存的前缀（绿色部分）中，草稿树令牌的KV状态将位于该前缀之后（黄色部分）。

模型和草稿树，并最大限度地重用先前计算的 KV 状态。

在整个执行过程中，我们按如下方式组织草稿模型的 KV 缓存，并在整个执行过程中保持其不变：已验证令牌的 KV 状态连续存储在 KV 缓存的前缀（我们称为前缀缓存）中，树的 KV 状态存储在前缀之后（我们称为树缓存）。

当目标工作人员对来自草稿工作人员的令牌进行批量推理和采样时，草稿工作人员不断在草稿树中生成新的树叶，将树缓存附加到现有条目之后。

重新组织已验证令牌的 KV 缓存 目标 Worker 对令牌进行采样后，会将已验证的令牌发送给草稿 Worker。然后，草案工作人员使用已验证的令牌遍历树，并在最后一个已验证的令牌处重新定位。然后，如果当前草稿树中存在最后一个验证的令牌，那么我们将重新组织树缓存，以便只有新子树中的节点的 KV 状态保留在树缓存中。这样，即使我们发送给目标 worker 的一些预测令牌是错误的，我们仍然可以重用子树中所有计算出的 KV 状态，避免任何重新计算。

图 5 显示了当有新的已验证令牌时如何更新草稿模型的 KV 缓存的示例。假设序列 (t_1, t_3, t_7, t_{10}) 已经被验证，并且缓存的前缀是这些令牌的 KV 状态，并且草案树令牌的 KV 状态在前缀之后连续组织。当我们将已验证的令牌更新为 $(t_1, t_3, t_7, t_{10}, t_{12}, t_{15})$ 时，我们使用新验证的令牌 (t_{12}, t_{15}) 沿着草稿树走下去。然后我们到达节点 t_{15} ，这意味着子树 t_{17}, t_{18} 中的节点在将来仍然有用。因此，我们将 t_{12}, t_{15} 移动到前缀缓存中，以便它存储与目标模型相同的已验证令牌的信息。然后它重新组织 t_{15} (的剩余子树，即 t_{17}, t_{18}) 进入下一个可用位置，丢弃不再有用的 KV 状态（例如 t_{11}, t_{12} 等）。

在草稿树没有足够的节点发送回目标工作程序的情况下，它会立即使用一个草稿模型推理来扩展 bs 节点。无论哪种情况，草稿树都会有足够的节点传递给目标工作人员，因此进入下一次迭代，并且草稿模型、目标模型和草稿树之间的 KV 状态同步。

3.3 延迟优化的内核设计

为了减少低批量大小下草稿和目标的推理时间，我们为 Llama 和 Qwen 模型系列设计并实现了延迟优化算子。虽然我们的设计可以应用于任何精度，但我们将其实现为 int4 AWQ 量化模型。我们优化的算子包括 all-reduce、masked Attention 和 SwiGLU。我们首先介绍 Nvidia 集体通信库的低延迟 (NCCL LL) 协议，我们的工作大量利用了该协议。

NCCL LL 协议。这是一种减少 GPU 到 GPU 发送和接收操作延迟的通信方案。算法 2 展示了 LL 协议的存储和加载功能。函数 AtomicStore 和 AtomicLoad 是 NVidia GPU PTX 指令 ld.volatile.global.v4.u32 和 st.volatile.global.v4.u32，它们在相同或不同的 GPU 中从全局内存加载和存储 16 字节数据。每个 8 字节（2 个 32 位无符号整数）加载/存储都是原子的。

storeLL 函数采用 64 位整数 val 和 32 位整数标志，将 val 拆分为两个 32 位整数，并将它们与 $flag$ 一起存储到内存位置。loadLL 函数采用一个内存位置和一个标志。它不断轮询，直到内存中的标志

```

1 Function
  storeLL(dst: address, val: 64-bit int, flag: 32-bit int):
2   val_low ← lower 32 bits of val;
3   val_high ← upper 32 bits of val;
4   AtomicStore(dst, val_low, flag, val_high,
    flag);
5 Function
  readLL(src: address, flag: 32-bit integer):
6   data1, flag1, data2, flag2 ← 0;
7   repeat
8     flag1, data1, flag2, data2 =
      AtomicLoad(src);
9   until;
10  flag1 = flag and flag2 = flag;
11  val64 ← data1 + ((uint64_t)data2 << 32);
12  return val64;
Algorithm 2: storeLL and readLL Function

```

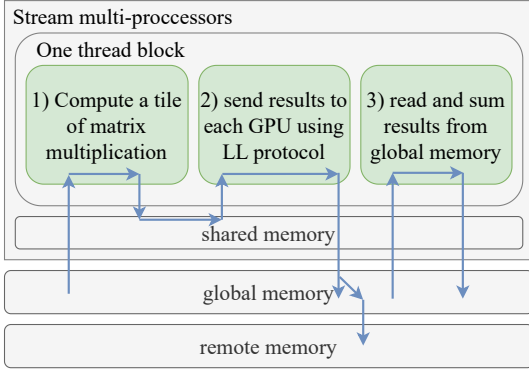


Figure 6 Execution flow of fused GEMM all-reduce operator

cation matches the expected one, and then combines the 2 32-bit integers into a 64-bit integer to return.

Using those two functions, we can implement a communication scheme without any explicit synchronization. Assume that last time we store some value x as a flag, and this time we use $x + 1$. The other compute unit (another stream multi-processor in the same GPU or on another GPU) will know the data is ready when it sees $x + 1$ as the flag, without any additional synchronization. In our latency-optimized kernels, we heavily rely on this powerful primitive to reduce synchronization overhead both within and across GPUs.

Fused GEMM with all reduce. To further reduce the data movement overhead and save the number of synchronization barriers, we fuse each all-reduce operation with the preceding GEMM operation. Fig-

ure 6 shows the computation and data flow within one thread block when we run GEMM fused with all reduce. Each thread block has three steps during execution:

- The threadblock calculates a contiguous set of columns in the final result matrix.
- The threadblock starts sending the result to each GPU using the NCCL LL protocol.
- The threadblock on each GPU waits until the results are available, aggregates them to get the final results, and stores them back to the global memory on the current GPU.

The synchronization between steps 2 and 3 is implemented using just the NCCL LL protocol described above and no additional, expensive mechanisms.

Masked attention. For the mask-attention operators, we fuse the position embedding with the attention calculation. Then, within one GPU, we first split the computation of the single attention head between different thread blocks. After the calculation, the threadblocks aggregate the sum using the NCCL LL protocol within a single GPU. Similar to our Fused GEMM with all-reduce operators, we add up the results within one attention head without explicit synchronization across thread blocks or extra kernel launches.

Fused SwiGLU. This operator is of the form $\text{SwiGLU}(x, W, V, b, c) = \sigma(xW + b) \oplus (xV + c)$. We implement tile-based matrix multiplication, where each threadblock calculates the same tile of the two matrix multiplications. This avoids loading the input twice from the GPU HBM. Right after we get the output of the tiles, we calculate the sigmoid and dot product before putting the results back to the GPU memory, avoiding unnecessary data movement.

4 Implementation

We implement our latency-optimized kernels using ~ 3000 lines of CUDA and C++, and our tree-based parallel speculative decoding using ~ 4000 lines of C++ and Python. We use the CUTLASS library [32] to implement the fused, latency-optimized kernels.

Aside from the optimized, fused all-reduce operators within the decoder layers in the transformer models, we use the NCCL library to communicate and synchronize states between different workers.

We wrap our optimized CUDA kernels in Python and capture all the kernels in a forward pass into

```

1 功能
  storeLL(dst: address, val: 64-bit int, flag: 32-bit int):
2    val_low ← lower 32 bits of val;
3    val_high ← upper 32 bits of val;
4    AtomicStore(dst, val_low, flag, val_high,
      flag);
5 Function
  readLL(src: address, flag: 32-bit integer):
6    data1, flag1, data2, flag2 ← 0;
7    repeat
8      flag1, data1, flag2, data2 =
        AtomicLoad(src);
9    until;
10   flag1 = flag and flag2 = flag;
11   val64 ← data1 + ((uint64_t)data2 << 32);
12   return val64;

```

Algorithm 2: storeLL and readLL Function

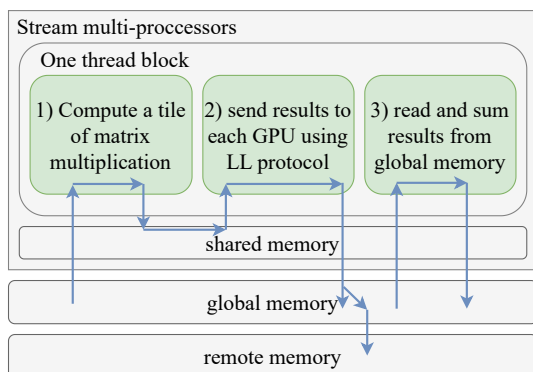


图6 融合GEMM全归约算子执行流程

cation 与预期匹配，然后将 2 个 32 位整数组合成 64 位整数返回。

使用这两个函数，我们可以实现无需任何显式同步的通信方案。假设上次我们存储一些值 x 作为标志，这次我们使用 $x + 1$ 。另一个计算单元（同一 GPU 或另一个 GPU 上的另一个流多处理器）在看到 $x + 1$ 作为标志时将知道数据已准备好，无需任何额外的同步。在我们的延迟优化内核中，我们严重依赖这个强大的原语来减少 GPU 内部和 GPU 之间的同步开销。

融合 GEMM 与全部归约。为了进一步减少数据移动开销并节省同步屏障的数量，我们将每个 all-reduce 操作与前面的 GEMM 操作融合在一起。如图-

图 6 显示了当我们运行与 allreduce 融合的 GEMM 时，一个线程块内的计算和数据流。每个线程块在执行过程中分为三个步骤：

- 线程块计算最终结果矩阵中的一组连续列。
- 线程块开始使用 NCCL LL 协议将结果发送到每个 GPU。
- 每个 GPU 上的线程块都会等待，直到结果可用，将它们聚合以获得最终结果，并将它们存储回当前 GPU 上的全局内存。

步骤 2 和步骤 3 之间的同步仅使用上述 NCCL LL 协议来实现，无需其他昂贵的机制。

蒙面关注。对于掩码注意力算子，我们将位置嵌入与注意力计算融合在一起。然后，在一个 GPU 内，我们首先将单个注意力头的计算拆分到不同的线程块之间。计算完成后，线程块在单个 GPU 内使用 NCCL LL 协议聚合总和。与带有 all-reduce 运算符的 Fused GEMM 类似，我们在一个注意力头内将结果相加，而无需跨线程块或额外的内核启动进行显式同步。

融合 SwiGLU。该运算符的形式为

$SwiGLU(x, W, V, b, c) = \sigma(xW + b) \oplus (xV + c)$ 。我们实现基于图块的矩阵乘法，其中每个线程块计算两个矩阵乘法的相同图块。这避免了从 GPU HBM 加载输入两次。在获得图块的输出后，我们立即计算 sigmoid 和点积，然后将结果放回 GPU 内存，从而避免不必要的数据移动。

4 实施

我们使用 ~ 3000 行 CUDA 和 C++ 实现延迟优化内核，并使用 ~ 4000 行 C++ 和 Python 实现基于树的并行推测解码。我们使用 CUTLASS 库 [32] 来实现融合的、延迟优化的内核。

除了 Transformer 模型中解码器层内的优化、融合的 all-reduce 算子之外，我们还使用 NCCL 库在不同工作线程之间进行通信和同步状态。

我们将优化的 CUDA 内核封装在 Python 中，并将前向传递中的所有内核捕获到

Target model	Draft model
Llama-3-70b-Instruct	Llama-3.2-3B, EAGLE-0.99B
deepseek-coder-33b-instruct	deepseek-coder-1.3b-instruct
Qwen2-72B-Instruct	Qwen2-1.5B-Instruct, EAGLE-1.05B
DeepSeek-R1-Distill-Qwen-32B	DeepSeek-R1-Distill-Qwen-1.5B
DeepSeek-R1-Distill-Llama-70B	DeepSeek-R1-Distill-Llama-8B

Table 4 The set of models we use in our evaluation of our approach and baselines

a CUDAGraph for both our draft model and target model to reduce kernel launch overhead.

Saving the number of CUDAGraphs for variable-sized masks. A CUDAGraph only supports a fixed-shape input. To reduce the number of CUDAGraphs that we need to capture, we perform a small optimization. We pad all tree masks to a fixed-size shape of $(w, maxseqlen)$, where w is the number of leaves to expand, and we copy the tree mask of size $(w, w + T)$, where T is the size of the tree, into the suffix of this larger structure and mask the prefix to be all 1’s. This allows us to reuse CUDAGraphs with variable-sized inputs. Specifically, we only need to capture $O(w_{max})$ (which is less than 20) CUDAGraphs, while supporting tree masks of more than one thousand different shapes.

Enabling arbitrary tensor parallelism for transformer models. Llama models (and transformer models from other families) usually need to satisfy two constraints before running tensor parallelism across x GPUs: (1) the dimension of the matrix multiplication (after split across x GPUs) is divisible by the GEMM kernel’s block size (usually 128 or 256); and (2) the number of attention heads must be divisible by x so that each attention head can be distributed to a single GPU. When we split the Llama 70B model across a power-of-two number of GPUs (i.e, 1/2/4/8), those two requirements are naturally satisfied since the number of attention heads and the dimension of the 70B model are divisible by a large power of 2.

However, when we want to enable a more fine-grained allocation of the draft model and the target model (e.g., using 6 GPUs for the target model and 2 GPUs for the draft model), those two requirements may not be satisfied. To resolve this, we increase the dimension of the matrix operation so that they are divisible by the number of GPUs x and zero-pad. For the attention calculation, we increase the number of attention heads so that it is divisible by x , and we also pad the projection matrix before and after the multi-head attention so that the calculation of padded attention heads does not contribute to the final result and the model output is equivalent of the

non-padded model.

5 Evaluation

In our evaluation, we want to answer and analyze the following questions:

- What is the end-to-end improvement of SwiftSpec compared to other speculative decoding baseline systems? (in §5.2)
- How much does parallel tree generation (with kv-cache synchronization in asynchronous decoding) improve the end-to-end performance? (in §5.3)
- How much does latency-optimized kernels improve the end-to-end performance? (in §5.3)
- What is the improvement of single operators using our latency-optimized kernels? (in §5.4)
- What are the design choices we make to achieve good performance using our framework? (in §5.5)

5.1 Experiments setup

Cluster setup We evaluate SwiftSpec and the baselines on one node with 8 NVIDIA 80GB H800 SXM GPUs connected by NVLink, which is the most common setup of a Nvidia Hopper compute node. We use all 8 GPUs to minimize the decoding latency for all of our experiments, except that, in §5.4, we show the performance improvement of our latency-optimized kernels under a subset of these GPUs.

Models and model configurations We evaluate our system under five different pairs of models (as shown in 4) from different families, including LLama3 [33], Deepseek-Coder [12], Qwen2 [38], Deepseek R1-Distilled Qwen, and Deepseek R1-Distilled Llama [9]. Deepseek-Coder is a series of models that focus on coding, while the rest of the model families have general abilities. From each model family, we pick a large model as the target model (generally having $> 30B$ parameters) and a small model as the draft model ($< 10B$). For Llama3 and Qwen2 family, there are also trained EAGLE2 models, so we used those in the baselines, which supports EAGLE-based [19] speculative decoding. We apply 4-bit AWQ quantization with a group size of 128 [20] to all the weights of the transformer layers models in each family except the EAGLE models. We keep the BF16 precision for the embedding layers and the LM head operator. Each model uses BF16 to compute the attention and the linear operators (after weight de-quantization).

Target model	Draft model
Llama-3-70b-Instruct	Llama-3.2-3B, EAGLE-0.99B
deepseek-coder-33b-instruct	deepseek-coder-1.3b-instruct
Qwen2-72B-Instruct	Qwen2-1.5B-Instruct, EAGLE-1.05B
DeepSeek-R1-Distill-Qwen-32B	DeepSeek-R1-Distill-Qwen-1.5B
DeepSeek-R1-Distill-Llama-70B	DeepSeek-R1-Distill-Llama-8B

表 4 我们在评估我们的方法和基线时使用的模型集

用于我们的草稿模型和目标模型的 CUDAGraph，以减少内核启动开销。

保存可变量大小掩码的 CUDAGraph 数量。CUDAGraph 仅支持固定形状输入。为了减少需要捕获的 CUDAGraph 数量，我们进行了一些小的优化。我们将所有树掩码填充为固定大小的形状 ($w, \maxseqlen$)，其中 w 是要扩展的叶子数量，然后将大小为 ($w, w + T$) 的树掩码（其中 T 是树的大小）复制到这个较大结构的后缀中，并将前缀掩码为全 1。这使我们能够重用具有可变量大小输入的 CUDAGraph。具体来说，我们只需要捕获小于 20) 个 CUDAGraph 的 $O(w_{max})$ (，同时支持一千多种不同形状的树掩码。

为变压器模型启用任意张量并行性。Llama 模型（以及其他系列的 Transformer 模型）在跨 x GPU 运行张量并行之前通常需要满足两个约束：(1) 矩阵乘法的维度（跨 x GPU 分割后）可被 GEMM 内核的块大小（通常为 128 或 256）整除；(2) 注意力头的数量必须能被 x 整除，以便每个注意力头可以分配到单个 GPU。当我们将 Llama 70B 模型划分为 2 的幂数的 GPU（即 1/2/4/8）时，这两个要求自然得到满足，因为注意力头的数量和 70B 模型的维度可以被 2 的大幂整除。

然而，当我们想要对草稿模型和目标模型进行更细粒度的分配时（例如，为目标模型使用 6 个 GPU，为草稿模型使用 2 个 GPU），这两个要求可能无法满足。为了解决这个问题，我们增加了矩阵运算的维度，以便它们可以被 GPU x 和零填充的数量整除。对于注意力计算，我们增加注意力头的数量，使其可以被 x 整除，并且我们还在多头注意力之前和之后填充投影矩阵，以便填充注意力头的计算对最终结果没有贡献，模型输出相当于

non-padded model.

5 评价

在我们的评估中，我们希望回答和分析以下问题：

- 与其他推测解码基线系统相比，Swift-Spec 的端到端改进是什么？（第 5.2 节）
- 并行树生成（在异步解码中使用 kv-cache 同步）对端到端性能提高了多少？（第 5.3 节）
- 延迟优化内核在多大程度上提高了端到端性能？（第 5.3 节）
- 使用我们的延迟优化内核的单个算子有何改进？（第 5.4 节）
- 为了使用我们的框架实现良好的性能，我们做出了哪些设计选择？（第 5.5 节）

5.1 实验设置

集群设置 我们在一个节点上评估 SwiftSpec 和基线，该节点具有通过 NVLink 连接的 8 个 NVIDIA 80GB H800 SXM GPU，这是 Nvidia Hopper 计算节点最常见的设置。我们使用全部 8 个 GPU 来最大限度地减少所有实验的解码延迟，但在第 5.4 节中，我们展示了延迟优化内核在这些 GPU 的子集下的性能改进。

模型和模型配置 我们在来自不同系列的五对不同模型（如图 4 所示）下评估我们的系统，包括 Llama3 [33]、Deepseek-Coder [12]、Qwen2 [38]、Deepseek R1-Distilled Qwen 和 Deepseek R1-Distilled Llama [9]。Deepseek-Coder 是一系列专注于编码的模型，而其余模型家族则具有一般能力。从每个模型系列中，我们选择一个大模型作为目标模型（通常具有 $> 30B$ 参数）和一个小模型作为草稿模型（ $< 10B$ ）。对于 Llama3 和 Qwen2 系列，也有经过训练的 EAGLE2 模型，因此我们在基线中使用了这些模型，它支持基于 EAGLE 的 [19] 推测解码。我们将组大小为 128 [20] 的 4 位 AWQ 量化应用于除 EAGLE 模型之外的每个系列中变压器层模型的所有权重。我们保持嵌入层和 LM 头算子的 BF16 精度。每个模型都使用 BF16 来计算注意力和线性算子（权重去量化之后）。

	EAGLE	smaller draft	draft TP	tree spec	performance
vLLM	✓	✓	✗	✗	not optimal
SGLang	✓	✗	✓	✓	optimal (EAGLE)
TRT-LLM	✓	✓	✓	✗	not optimal
SwiftSpec-base	✗	✓	✓	✓	optimal (smaller draft)

Table 5 The support matrix for different baselines in the evaluation under int4 AWQ

We apply the same quantization to both the baseline and our system, and therefore, even with the latency-optimized kernels, the computation of each single model in our system is equivalent to that of each baseline model.

Datasets We evaluated our system in six different datasets across different domains: MT-bench [43], HumanEval [7], GSM8K [8], Alpaca [31], CNN/Daily Mail [26], and Natural Questions [16]. The datasets consist of tasks including summarization, math questions, coding questions, etc.

Baseline systems We compare our solution against three major LLM serving frameworks: vLLM, SGLang, and TensorRT-LLM.

- vLLM [17]: vLLM supports both EAGLE and a smaller model from the same model family as the draft model. However, it only supports sequence-based speculative decoding for both cases.
- SGLang [44]: SGLang supports using the EAGLE model as the draft model, with tree-based generation. Out of the five model families we benchmark on, only two pairs (Llama 70B and Qwen 72B) have the corresponding EAGLE draft model, and therefore, we benchmark the speed of the normal auto-regressive generation for the other three model families.
- TRT-LLM [27]: Under int4-awq precision, TRT-LLM only supports sequence-based speculative decoding with a smaller draft model.

Since the normal tree-based speculative decoding with tensor parallelism is not supported on vLLM and TRT-LLM, to make a fair comparison with the existing techniques, we also implement:

- SwiftSpec-base: This is an implementation with only our optimized custom attention kernels. It uses the one-shot all-reduce implementation in the PyTorch Distributed library, and a SwiGLU implementation using normal AWQ multiplication kernel and PyTorch implementation of *silu* activation and dot product. It also performs sequential speculative decoding with the draft model and the target model both running on all

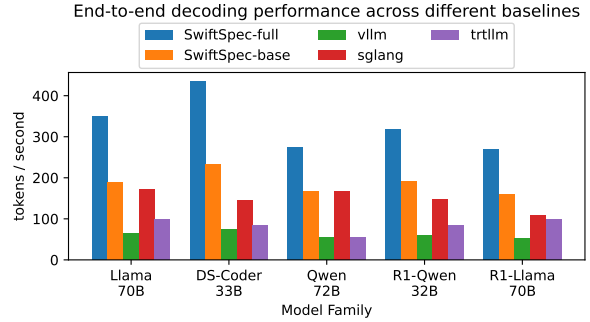


Figure 7 End-to-end decoding speed comparison between baselines.

GPUs.

Table 5 shows the configuration of speculative decoding that each baseline supports. As shown in later benchmarks, the most competitive baselines are SGLang and SwiftSpec-base since they support serial tree-based speculative decoding for EAGLE and a smaller draft, respectively. For each baseline implementation of each model, we run them in an extensive set of configurations and choose the configuration that maximizes the average tokens per second across all the datasets.

5.2 End-to-end results

Figure 7 shows the end-to-end benchmark on the decoding speed of SwiftSpec along with the four baselines, vLLM, SGLang, TRT-LLM, and SwiftSpec-base. Due to the limitation that vLLM and TRT-LLM only support sequence-based speculative decoding, their performance is not comparable with SGLang and SwiftSpec-base. For the Llama 70B model and the Qwen 72B model, EAGLE speculative decoding is supported in SGLang, and thus SGLang has a comparable performance with SwiftSpec-based; for the other models (Deepseek-Coder, R1-Qwen, and R1-Llama), we benchmark SGLang with autoregressive decoding since EAGLE and other speculative decoding methods are not supported. Therefore, for those models, SwiftSpec-base serves as a baseline that we can compare more fairly. As shown in Figure 7, our SwiftSpec system consistently outperforms SwiftSpec-base (by an average of $1.75\times$) and SGLang (on average $2.23\times$), the two most competitive baselines in the benchmark.

5.3 Ablation studies

In this paper, we propose three techniques, which help two classes of optimizations: Parallel tree generation and consistency management together support efficient parallel tree-based speculative decoding, while

	EAGLE	smaller draft	draft TP	tree spec	performance
vLLM	✓	✓	×	×	not optimal
SGLang	✓	×	✓	✓	optimal (EAGLE)
TRT-LLM	✓	✓	✓	×	not optimal
SwiftSpec-base	×	✓	✓	✓	optimal (smaller draft)

表5 int4 AWQ评估中不同基线的支持矩阵

我们对基线和我们的系统应用相同的量化，因此，即使使用延迟优化的内核，我们系统中每个模型的计算也相当于每个基线模型的计算。

数据集 我们在不同领域的六个不同数据集中评估了我们的系统：MT-bench [43]、HumanEval [7]、GSM 8K [8]、Alpaca [31]、CNN/Daily Mail [26] 和 Natural Questions [16]。数据集由总结、数学问题、编码问题等任务组成。

基线系统 我们将我们的解决方案与三个主要的 LLM 服务框架进行比较：vLLM、SGLang 和 TensorRT-LLM。

- vLLM [17]: vLLM 支持 EAGLE 和与草案模型相同的模型系列中的较小模型。但是，它仅支持这两种情况的基于序列的推测解码。
- SGLang [44]: SGLang 支持使用 EAGLE 模型作为草稿模型，并基于树生成。在我们进行基准测试的五个模型系列中，只有两对（Llama 70B 和 Qwen 72B）具有相应的 EAGLE 草案模型，因此，我们对其他三个模型系列的正常自回归生成的速度进行了基准测试。
- TRT-LLM [27]: 在 int4-awq 精度下，TRT-LLM 仅支持具有较小草稿模型的基于序列的推测解码。

由于 vLLM 和 TRT-LLM 不支持具有张量并行性的普通基于树的推测解码，为了与现有技术进行公平比较，我们还实现了：

- SwiftSpec-base: 这是一个仅使用我们优化的自定义注意力内核的实现。它使用 PyTorch 分布式库中的一次性全归约实现，以及使用正常 AWQ 乘法内核的 SwiGLU 实现以及 *silu* 激活和点积的 PyTorch 实现。它还对草稿模型和目标模型执行顺序推测解码，这些模型都运行在所有机器上。

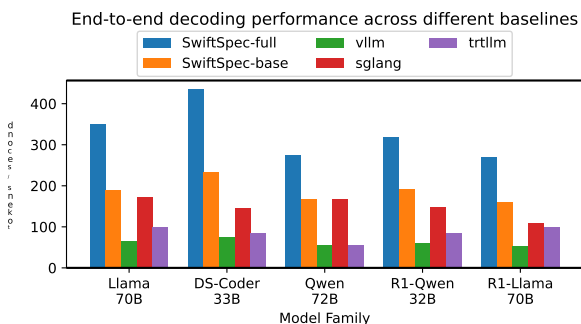


图 7 基线之间的端到端解码速度比较。

GPU。

表 5 显示了每个基线支持的推测解码的配置。如后面的基准测试所示，最具竞争力的基线是 SGLang 和 SwiftSpec-base，因为它们分别支持 EAGLE 和较小草案的基于串行树的推测解码。对于每个模型的每个基线实现，我们在一组广泛的配置中运行它们，并选择最大化所有数据集每秒平均令牌的配置。

5.2 端到端结果

图 7 显示了 SwiftSpec 解码速度的端到端基准测试以及四个基准：vLLM、SGLang、TRT-LLM 和 SwiftSpec-base。由于 vLLM 和 TRT-LLM 仅支持基于序列的推测解码的限制，它们的性能无法与 SGLang 和 SwiftSpec-base 相媲美。对于 Llama 70B 模型和 Qwen 72B 模型，SGLang 支持 EAGLE 推测解码，因此 SGLang 具有与基于 SwiftSpec 相当的性能；对于其他模型（Deepseek-Coder、R1-Qwen 和 R1-Llama），我们使用自回归解码对 SGLang 进行基准测试，因为不支持 EAGLE 和其他推测性解码方法。因此，对于这些模型，SwiftSpec-base 作为我们可以更公平比较的基准。如图 7 所示，我们的 SwiftSpec 系统始终优于 SwiftSpec-base（平均 1.75×）和 SGLang（平均 2.23×），这两个基准中最具竞争力的基线。

5.3 消融研究

在本文中，我们提出了三种技术，有助于两类优化：并行树生成和一致性管理共同支持高效的并行基于树的推测解码，而

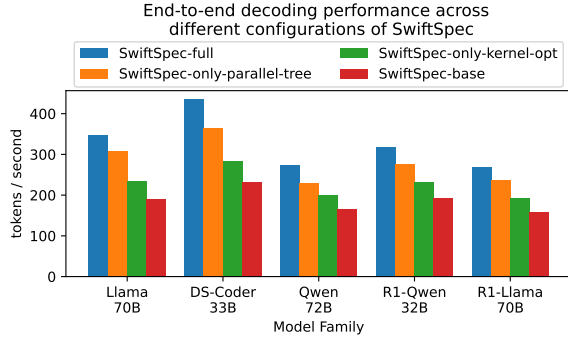


Figure 8 End-to-end decoding speed comparison across different configurations of SwiftSpec

latency-optimized kernels support more efficient kernel execution. We study the effect of each of those two classes of optimizations to reason about where our performance gain comes from.

As shown in Figure 8, we benchmark our framework under four different configurations of our framework: ① SwiftSpec, ② SwiftSpec-only-parallel-tree, ③ SwiftSpec-only-kernel-opt, ④ SwiftSpec-base. SwiftSpec and SwiftSpec-base are introduced earlier; SwiftSpec-only-parallel-tree is a version of SwiftSpec with only the parallel tree generation mechanism and the optimized attention kernels, while SwiftSpec-only-kernel-opt is a version of SwiftSpec with only the latency-optimized kernels (and serial speculative decoding). Note that all four configurations contain our customized attention kernels since a non-square mask is needed to support our tree expansion algorithm, which is not supported by other works. We reason about the improvement of our attention kernel optimization alone later in this section and also in the kernel micro-benchmark section (§5.4).

Effect of parallel tree generation (with KV synchronization) To show the effect of our parallel tree generation with KV cache consistency management, we compare ① with ③ (using vs not using parallel tree generation under latency-optimized kernels) and ② with ④ (using vs not using parallel tree generation without latency-optimized kernels). As shown in Figure 8, ① outperforms ③ by an average speedup of $1.43\times$, and ② outperforms ④ by an average speed of $1.50\times$.

To look deeper into how our parallel tree generation mechanism brings an improvement of an average of 43% (from ③ to ①), we look specifically into the Qwen2 model family as an example, where the optimal configurations (including tree depth d , tree width w , etc) are the same for using and not using parallel tree generation. While the serial version uses

	Decoding speed	model time		compression ratio for different datasets						
		target	draft	ALP	GSM	HE	MT	QA	SUM	
parallel	275 tokens/s	10.48ms	3.25ms	2.92	3.78	3.92	3.11	2.74	3.56	
serial	200 tokens/s	10.34ms	3.73ms	3.28	4.2	4.04	3.42	3.12	3.72	

Table 6 The comparison of compression ratio, model inference time, and decoding speed under parallel and serial tree generation for model Qwen2-72B and draft model Qwen2-1.5B. ALP, GSM, etc are the abbreviations of the six datasets we use in the benchmark.

8 GPUs for both models, the parallel version uses 2 GPUs for the draft model and 6 GPUs for the target model.

Table 6 shows the compression ratio (average number of successful guesses per target model verification) and the target and draft model inference time under parallel/serial speculative decoding (① and ③). When we are running parallel tree generation instead of serial speculative decoding, the draft model does not know the result of the concurrent target model verification, and thus it generates some nodes that are not useful after the verification. Therefore, the compression ratio is smaller when we use parallel tree generation. However, due to our maximum-likelihood expansion of the draft tree and the maximized reuse of the KV cache, the compression ratio is only 9% less than the serial version on average. Furthermore, our draft model inference time decreases from $3.72ms$ to $3.25ms$ using only 2 GPUS instead of 8 GPUs saving 79% of the GPU cycles during one round of drafting, and our target model has nearly no slow down using 6 GPUs instead of 8 GPUs (from $10.34ms$ to $10.48ms$), still saving 25% of the GPUs cycles during one round of target verification. As a result, the end-to-end tokens-per-second of Qwen2-72B model increases from 200 to 275 (by 37%) by applying our parallel tree generation and KV cache management scheme.

Effect of latency-optimized kernels To illustrate effect of our latency-optimized fused GEMM-all-reduce kernel and SwiGLU operators, we compare ① with ② (using vs not using those kernels under parallel tree generation) and ③ with ④ (using vs not using those kernels under serial tree generation). As shown in Figure 8, ① outperforms ② by an average speedup of $1.16\times$, and ③ outperforms ④ by an average of $1.21\times$. Therefore, those two kernel optimizations give the end-to-end improvement of at least 16% for both parallel and serial tree generation.

To illustrate the improvement of our optimized attention kernel, we benchmark the Llama 3B model under TP=4 as an example. Under an input token length of 8 and a context length of 500, an inference

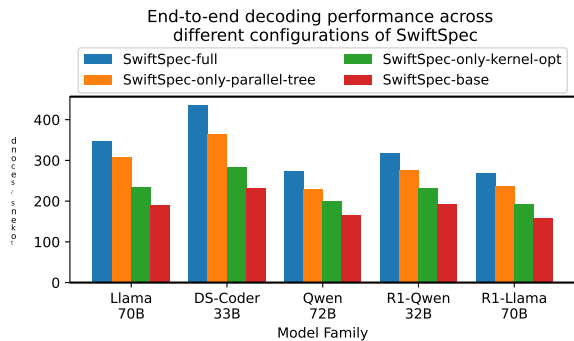


图8 SwiftSpec不同配置的端到端解码速度对比

延迟优化内核支持更高效的内核执行。我们研究这两类优化的效果，以推断我们的性能增益来自何处。

如图 8 所示，我们在框架的四种不同配置下对框架进行了基准测试：1 SwiftSpec、2 SwiftSpec-only-parallel-tree、3 SwiftSpec-only-kernel-opt、4 SwiftSpec-base。前面介绍过 SwiftSpec 和 SwiftSpec-base；SwiftSpec-only-parallel-tree 是仅具有并行树生成机制和优化注意力内核的 SwiftSpec 版本，而 SwiftSpec-only-kernel-opt 是仅具有延迟优化内核（和串行推测解码）的 SwiftSpec 版本。请注意，所有四种配置都包含我们定制的注意内核，因为需要非方形掩码来支持我们的树扩展算法，而其他作品不支持该算法。我们将在本节后面以及内核微基准测试部分（第 5.4 节）中单独对注意力内核优化的改进进行推理。

并行树生成（使用 KV 同步）的效果 为了展示使用 KV 缓存一致性管理的并行树生成的效果，我们将 1 与 3（在延迟优化内核下使用与不使用并行树生成）和 2 与 4（在没有延迟优化内核的情况下使用与不使用并行树生成）进行比较。如图 8 所示，1 的平均加速比 3 好 1.43 $\times$ ，2 的平均速度比 4 好 1.50 $\times$ 。

为了更深入地了解我们的并行树生成机制如何带来平均 43% 的改进（从 3 到 1），我们特别以 Qwen2 模型系列为例，其中使用和不使用并行树生成时的最佳配置（包括树深度 d 、树宽度 w 等）是相同的。虽然串行版本使用

	Decoding speed	model time		compression ratio for different datasets					
		target	draft	ALP	GSM	HE	MT	QA	SUM
parallel	275 tokens/s	10.48ms	3.25ms	2.92	3.78	3.92	3.11	2.74	3.56
serial	200 tokens/s	10.34ms	3.73ms	3.28	4.2	4.04	3.42	3.12	3.72

表6 模型Qwen2-72B和草案模型Qwen2-1.5B在并行和串行树生成下的压缩率、模型推理时间和解码速度比较。ALP、GSM 等是我们在基准测试中使用的六个数据集的缩写。

两个模型都有 8 个 GPU，并行版本使用 2 个 GPU 用于草稿模型，6 个 GPU 用于目标模型。

表 6 显示了并行/串行推测解码（1 和 3）下的压缩比（每次目标模型验证成功猜测的平均次数）以及目标和草稿模型推理时间。当我们运行并行树生成而不是串行推测解码时，草稿模型不知道并发目标模型验证的结果，因此会生成一些验证后无用的节点。因此，当我们使用并行树生成时，压缩率较小。然而，由于我们对草稿树进行了最大似然扩展，并且最大限度地复用了KV缓存，因此压缩率平均只比串行版本低9%。此外，我们的草稿模型推理时间从 3.72ms 减少到 3.25ms，仅使用 2 个 GPU 而不是 8 个 GPU，在一轮草稿期间节省了 79% 的 GPU 周期，并且我们的目标模型使用 6 个 GPU 而不是 8 个 GPU 几乎没有减慢速度（从 10.34ms 到 10.48ms），仍然节省一轮目标验证期间 25% 的 GPU 循环。结果，通过应用我们的并行树生成和 KV 缓存管理方案，Qwen2-72B 模型的每秒端到端令牌数从 200 增加到 275（增加 37%）。

延迟优化内核的效果 为了说明我们的延迟优化融合 GEMM-all-reduce 内核和 SwiGLU 算子的效果，我们比较 1 与 2（在并行树生成下使用与不使用这些内核）以及 3 与 4（在串行树生成下使用与不使用这些内核）。如图 8 所示，1 比 2 平均加速 1.16 $\times$ ，3 比 4 平均加速 1.21 $\times$ 。因此，这两个内核优化为并行和串行树生成提供了至少 16% 的端到端改进。

为了说明我们优化的注意力内核的改进，我们以 TP=4 下的 Llama 3B 模型为例进行基准测试。在输入标记长度为 8 且上下文长度为 500 的情况下，推理

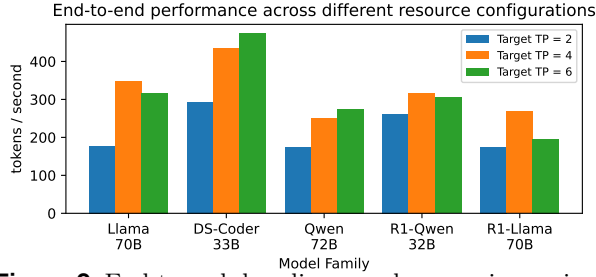


Figure 9 End-to-end decoding speed comparison using different resource configurations for draft model and target model

of the 3B model with all the latency-optimized kernel optimizations takes 3.25 ms to finish, while an inference with a FlashAttention kernel takes 3.54 ms. This shows an 8.2% reduction in the inference time of the Llama 3B model using our optimized attention kernel.

Refer to §5.4 for the inference time of individual operators to understand our per-operator improvement.

5.4 Latency-optimized operator microbenchmark

In Table 7, we show the individual kernel times of our latency-optimized kernels and kernels in other baselines. We focus on the Llama3 model family as a specific example, while our optimizations are also applied to other model families in our benchmark, achieving similar speedup.

Fused GEMM with all reduce In each layer of the Llama3 model under tensor parallelism, there are two all-reduce operations, one in the attention block, and the other in the MLP block. We fuse each all-reduce operation with the previous GEMM. In the implementation of both vllm and TRTLLM, they use their implementation of one-shot all-reduce, and they perform the all-reduce operations after the GEMM operations as separate kernels. We sum up the time spent on two kernels as their total time. In our solution, we fuse the GEMM operation with our implementation of one-shot all-reduce. As shown in Table 7, our fused GEMM-all-reduce kernel consistently outperforms vllm and TRTLLM on all five model configurations. Our improvement is larger when the amount of compute is less: this happens for the fused GEMM in attention block for all models (with a time reduction of 23%-43%) and that in MLP block for the smaller models, 1B, 3B, and 8B (with a time reduction of 16%-25%).

Attention operator For the attention operator, we compare our implementation with two popular atten-

tion libraries: FlashAttention (FA) and FlashInfer (FI). To make a fair comparison to FA and FI, which only support square kernels, during the benchmark, we only feed square masks to our tree attention kernel, even though it supports a more general non-square mask. We use BatchPrefillWithPagedKVCacheWrapper in FlashInfer, which supports a square custom mask. However, the kernel is not optimized for small numbers of attention heads, and, therefore, performs much worse than the other baseline, FA. FA uses kernel `flash_fwd_splitkv_kernel` to split each kv head across different threadblocks to compute the attention score and sum faster, and then it launches another kernel `flash_fwd_splitkv_combine_kernel` to aggregate the results across different threadblocks. On the other hand, our kernel fuses those two kernels into one, using the NCCL LL protocol to perform an implicit barrier, saving time for the overhead of synchronization and an extra kernel launch. As shown in Table 7, our optimization kernel constantly saves 30% to 56% of the running time compared to FA under two representative context lengths, 500 and 1000, across different model configurations.

SwiGLU operator In each layer of the Llama3 model, we fuse the four operations in the SwiGLU operator ($\text{SwiGLU}(x, W, V, b, c) = \sigma(xW + b) \oplus (xV + c)$) into one, reducing data movement and extra kernel launches. Both vllm and TRTLLM fuse the dot product operator with the σ activation operator. Furthermore, vllm fuses the first two matrix multiplications into one operator. As shown in Table 7, our SwiGLU optimization outperforms other baselines (with a time reduction of 39%-50%) when the model is small (1B, 3B), where latency dominates the whole execution time. When the model is larger and the amount of computation is more (for the 70B model), TRTLLM and vllm outperform our kernel since they have more optimized kernels (including AWQ marlin, where a more intricate layout of weight matrices is used to improve performance), while our kernel is based on simple tile-based GEMM and uniform layout.

5.5 Design choices

Choice about target batch size, draft batch size, depth In section 2.3, Figure 2 shows that increasing the draft model batch size over 8 only increases the compression ratio marginally. For the target model, there is still an average increase of 24.5% when we increase the batch size from 8 to 16. However, for example, if we increase the batch size of the Llama3-70B model from 8 to 16 under TP=4, the inference time will increase from 10.39 ms to 13.25 ms (by 29%). Thus, the increase in compression ratio does

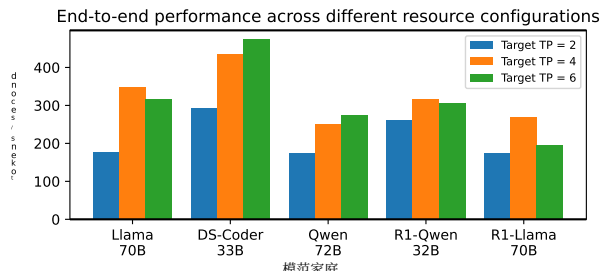


图 9 草稿模型和目标模型不同资源配置的端到端解码速度对比

具有所有延迟优化内核优化的 3B 模型的推理需要 3.25 毫秒才能完成，而使用 FlashAttention 内核的推理则需要 3.54 毫秒。这表明使用我们优化的注意力内核，Llama 3B 模型的推理时间减少了 8.2%。

请参阅第 5.4 节了解各个算子的推理时间，以了解我们每个算子的改进。

5.4 延迟优化的算子微基准

在表 7 中，我们显示了延迟优化内核和其他基线中的内核的各个内核时间。我们重点关注 Llama3 模型系列作为具体示例，同时我们的优化也应用于基准测试中的其他模型系列，实现了类似的加速。

与all-reduce融合的GEMM在张量并行下的Llama3模型的每一层中，有两个all-reduce操作，一个在attention块中，另一个在MLP块中。我们将每个all-reduce操作与之前的GEMM融合。在vllm和TRTLLM的实现中，它们都使用一次性all-reduce的实现，并且在GEMM操作之后作为单独的内核执行all-reduce操作。我们将两个内核花费的时间相加作为它们的总时间。在我们的解决方案中，我们将GEMM操作与一次性全归约的实现融合在一起。如表 7 所示，我们的融合GEMM-all-reduce内核在所有五种模型配置上始终优于vllm和TRTLLM。当计算量较少时，我们的改进更大：这种情况发生在所有模型的注意力块中的融合GEMM中（时间减少了23%-43%），以及较小模型1B、3B和8B的MLP块中的融合GEMM（时间减少了16%-25%）。

注意力运算符对于注意力运算符，我们将我们的实现与两种流行的注意力进行比较

化库：FlashAttention (FA) 和 FlashInfer (FI)。为了与只支持方形内核的 FA 和 FI 进行公平比较，在基准测试期间，我们只将方形掩码提供给我们树注意力内核，即使它支持更通用的非方形掩码。我们在FlashInfer中使用BatchPrefillWithPagedKVCacheWrapper，它支持方形自定义掩码。然而，内核并未针对少量注意力头进行优化，因此，其性能比其他基线 FA 差得多。FA 使用内核 flash_fwd_splitkv_kernel 将每个 kv 头拆分到不同的线程块上，以更快地计算注意力分数和求和，然后启动另一个内核 flash_fwd_splitkv_combine_kernel 来聚合不同线程块上的结果。另一方面，我们的内核将这两个内核融合为一个，使用 NCCL LL 协议执行隐式屏障，从而节省同步开销和额外内核启动的时间。如表 7 所示，在不同的模型配置下，在两个代表性上下文长度 500 和 1000 下，我们的优化内核与 FA 相比不断节省 30% 到 56% 的运行时间。

SwiGLU 运算符在 Llama3 模型的每一层中，我们将 SwiGLU 运算符 (

$SwiGLU(x, W, V, b, c) = \sigma(xW + b) \oplus (xV + c)$) 中的四个操作融合为一个，从而减少数据移动和额外的内核启动。vllm 和 TRTLLM 都将点积运算符与 σ 激活运算符融合。此外，vllm 将前两个矩阵乘法融合为一个运算符。如表 7 所示，当模型较小 (1B、3B) 时，延迟在整个执行时间中占主导地位，我们的 SwiGLU 优化优于其他基线（时间减少了 39%-50%）。当模型较大且计算量较多时（对于 70B 模型），TRTLLM 和 vllm 优于我们的内核，因为它们具有更优化的内核（包括 AWQ marlin，其中使用更复杂的权重矩阵布局来提高性能），而我们的内核基于简单的基于 tile 的 GEMM 和统一布局。

5.5 设计选择

关于目标批量大小、草稿批量大小、深度的选择在第 2.3 节中，图 2 显示将草稿模型批量大小增加到超过 8 仅会略微增加压缩比。对于目标模型，当我们将批量大小从 8 增加到 16 时，仍然平均增加了 24.5%。但是，例如，如果我们在 TP=4 下将 Llama3-70B 模型的批量大小从 8 增加到 16，则推理时间将从 10.39 ms 增加到 13.25 ms（增加了 29%）。因此，压缩比的增加

	Fused GEMM (attn)			Fused GEMM (mlp)			SwiGLU			Attn, len(context)=500			Attn, len(context)=1000		
	Ours	vllm	TRT	Ours	vllm	TRT	Ours	vllm	TRT	Ours	FI	FA	Ours	FI	FA
1B, tp = 4	5.9us	12.9us	10.2us	8.3us	18.8us	11.0us	5.8us	12.3us	11.5us	6.3us	13.6us	13.8us	6.2us	22.8us	13.8us
3B, tp = 4	6.4us	12.3us	10.0us	8.7us	17.0us	10.3us	7.2us	12.5us	11.8us	6.2us	19.1us	17.2us	8.0us	33.4us	18.3us
8B, tp = 4	7.7us	14.6us	10.3us	10.5us	16.5us	11.9us	15.0us	16.3us	15us	6.4us	19.4us	17.75us	8.1us	33.4us	18.13us
70B, tp = 4	11.72us	16.9us	15.5us	24.3us	25.4us	26.1us	49us	36.7us	31.9us	9.6us	19.2us	19.0us	13.4us	33.2us	19.1us
70B, tp = 8	13.3us	25.7us	17.2us	19.9us	29.5us	20.7us	23.6us	22.5us	22us	6.4us	18.6us	17.8us	8.1us	32.7us	18.2us

Table 7 Time per kernel of our optimized operator under batch size 8 across different TP configurations in the Llama family.

not cover the increase in inference time. Furthermore, reducing the batch size to under 8 does not reduce the inference time (e.g. It takes 10.32ms under batch size 4) since the smallest first input dimension of the matrix multiplication tensor core operation is 8. As a result, we use batch size = 8 for both our target model and draft model.

To choose d , the number of tree expansions each round, as reasoned in §3.1, we choose one of the two integers that are closest to the time ratio of a target model inference and one round of draft tree expansion. In the benchmark, for each model pair, we run SwiftSpec using those two different d s based on the draft and target model execution time and choose the configuration with the higher decoding speed.

Draft model and target model resource allocation.

Figure 9 illustrates how flexible allocation of GPU resources between draft and target models affects overall decoding performance. Specifically, it shows the performance of each model family when we allocate different numbers of GPUs to the target and draft models. We only consider TP=2,4,6 for the target model since even degree of tensor parallelism is more aligned to the attention operators and matrix multiplications and thus requires less padding. For Deepseek-Coder 33B and Qwen2-72B, we find it optimal to use TP=6 for the target model and TP=2 for the draft model, and for other models, it is the best to use TP=4 for both models since for those model pairs, giving more compute to a more capable draft model can increase the number of tree layers and thus increase the number of accepted tokens per round.

6 Discussion and Limitations

EAGLE speculative decoding vs smaller-draft speculative decoding

While SwiftSpec could make normal speculative decoding with a smaller draft model faster, it is non-trivial to apply them into EAGLE speculative decoding, since the EAGLE model takes the target model output of the verified tokens as input, imposing a even stricter dependency between

draft model and target model that is hard to break. Therefore, our approach is more suitable when an "independent" draft model, preferably a smaller model from the same model family, is available. How to integrate our method with EAGLE is left as future work.

High-throughput serving Our use of int4 quantized model and its latency-optimized kernels are naturally not suitable for high-throughput serving scenarios where the batch size is larger (e.g. ≥ 64). However, we believe our parallel tree generation algorithm could still be useful for larger batch sizes (especially when the GPU interconnect is slow) since it effectively decreases the amount of communication across different GPUs. However, as speculative decoding is generally useful when the workload of the system is relatively light [22], when the system workload is high and it is more cost-efficient to run autogressive decoding, our approach, based on speculative decoding, becomes less effective as well.

Disaggregated execution LLM serving systems such as [29, 45] use the idea of disaggregating the prefill phase and the decoding phase. SwiftSpec’s parallel tree generation idea is similar in spirit to those approaches, allowing different types of computation to scale independently. On the other hand, SwiftSpec focuses on disaggregating different components in the decoding phase and is thus orthogonal to those approaches. Therefore, approaches that accelerate the prefill phase [23, 42] are complementary to our system.

7 Conclusion

Increasing the decoding speed of large language models by scaling speculative decoding is important yet challenging for reducing end-to-end request latency. This paper proposes SwiftSpec, which includes three key innovations: *parallel tree generation*, *consistent KV cache management*, and *latency-optimized kernels*. Combining those three techniques, we are able to serve five different models in different model families with an average decoding speed $1.75\times$ faster than the most competitive baselines. Particularly,

	Fused GEMM (attn)			Fused GEMM (mlp)			SwiGLU			Attn, len(context)=500			Attn, len(context)=1000		
	Ours	vllm	TRT	Ours	vllm	TRT	Ours	vllm	TRT	Ours	FI	FA	Ours	FI	FA
1B, tp = 4	5.9us	12.9us	10.2us	8.3us	18.8us	11.0us	5.8us	12.3us	11.5us	6.3us	13.6us	13.8us	6.2us	22.8us	13.8us
3B, tp = 4	6.4us	12.3us	10.0us	8.7us	17.0us	10.3us	7.2us	12.5us	11.8us	6.2us	19.1us	17.2us	8.0us	33.4us	18.3us
8B, tp = 4	7.7us	14.6us	10.3us	10.5us	16.5us	11.9us	15.0us	16.3us	15us	6.4us	19.4us	17.75us	8.1us	33.4us	18.13us
70B, tp = 4	11.72us	16.9us	15.5us	24.3us	25.4us	26.1us	49us	36.7us	31.9us	9.6us	19.2us	19.0us	13.4us	33.2us	19.1us
70B, tp = 8	13.3us	25.7us	17.2us	19.9us	29.5us	20.7us	23.6us	22.5us	22us	6.4us	18.6us	17.8us	8.1us	32.7us	18.2us

表 7 Llama 系列中不同 TP 配置的批量大小为 8 时优化算子的每个内核时间。

不包括推理时间的增加。此外，将批量大小减少到 8 以下并不会减少推理时间（例如，在批量大小 4 下需要 10.32ms），因为矩阵乘法张量核心运算的最小第一输入维度是 8。因此，我们对目标模型和草稿模型都使用批量大小 = 8。

为了选择每轮树扩展的数量 d ，如 §3.1 中的推理，我们选择最接近目标模型推理和一轮草图树扩展的时间比率的两个整数之一。在基准测试中，对于每个模型对，我们根据草稿和目标模型执行时间使用这两个不同的 d 运行 SwiftSpec，并选择具有更高解码速度的配置。

草案模型和目标模型资源分配。图 9 说明了草稿模型和目标模型之间 GPU 资源的灵活分配如何影响整体解码性能。具体来说，它显示了当我们为目标模型和草稿模型分配不同数量的 GPU 时每个模型系列的性能。我们只考虑 TP=2,4,6 作为目标模型，因为即使张量并行度也更符合注意力算子和矩阵乘法，因此需要更少的填充。对于 Deepseek-Coder 33B 和 Qwen2-72B，我们发现最好对目标模型使用 TP=6，对草案模型使用 TP=2，而对于其他模型，最好对两个模型使用 TP=4，因为对于这些模型对，为能力更强的草案模型提供更多计算可以增加树层数，从而增加每轮接受的令牌数量。

6 讨论和局限性

EAGLE 推测性解码与较小草稿推测性解码虽然 SwiftSpec 可以使较小草稿模型的正常推测性解码更快，但将它们应用到 EAGLE 推测性解码中并非易事，因为 EAGLE 模型将已验证令牌的目标模型输出作为输入，从而在两者之间施加了更严格的依赖关系。

草案模型和目标模型很难打破。因此，当“独立”草图模型（最好是来自同一模型系列的较小模型）可用时，我们的方法更合适。如何将我们的方法与 EAGLE 集成是未来的工作。

高吞吐量服务我们使用 int4 量化模型及其延迟优化内核自然不适合批量大小较大的高吞吐量服务场景（例如 ≥ 64 ）。然而，我们相信我们的并行树生成算法对于较大的批量大小仍然有用（特别是当 GPU 互连速度较慢时），因为它有效地减少了不同 GPU 之间的通信量。然而，由于推测解码通常在系统工作负载相对较轻时有用[22]，当系统工作负载较高且运行自回归解码更具成本效益时，我们基于推测解码的方法也变得不太有效。

分解执行 LLM 服务系统（例如 [29, 45]）使用分解预填充阶段和解码阶段的思想。SwiftSpec 的并行树生成思想在本质上与这些方法类似，允许不同类型的计算独立扩展。另一方面，SwiftSpec 专注于在解码阶段分解不同的组件，因此与这些方法正交。因此，加速预填充阶段的方法 [23, 42] 是对我们系统的补充。

7 结论

通过扩展推测解码来提高大型语言模型的解码速度对于减少端到端请求延迟来说很重要，但也具有挑战性。本文提出了 SwiftSpec，其中包括三个关键创新：并行树生成、一致的 KV 缓存管理和延迟优化内核。结合这三种技术，我们能够为不同模型系列中的五种不同模型提供服务，平均解码速度比最具竞争力的基线快 1.75 $\times$ 。特别，

SwiftSpec can serve the Llama3 70B model on an average of 348 tokens per second, which is currently the fastest system using Nvidia Hopper GPUs. We believe the parallel tree generation with KV cache consistency and fused operator strategies are general and independent of the particular model families we test, so we expect these key techniques SwiftSpec to generalize to future speculative decoding architectures.

SwiftSpec 可以平均每秒 348 个令牌为 Llama3 70B 模型提供服务，这是目前使用 Nvidia Hopper GPU 的最快系统。我们相信具有 KV 缓存一致性和融合运算符策略的并行树生成是通用的，并且独立于我们测试的特定模型系列，因此我们期望 SwiftSpec 这些关键技术能够推广到未来的推测解码架构。

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- [2] Branden Butler, Sixing Yu, Arya Mazaheri, and Ali Jannesari. Pipeinfer: Accelerating llm inference using asynchronous pipelined speculation, 2024. URL <https://arxiv.org/abs/2407.11798>.
- [3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL <https://arxiv.org/abs/2401.10774>.
- [4] Kevin Matthe Caramancion. Large language models vs. search engines: Evaluating user preferences across varied information retrieval scenarios, 2024. URL <https://arxiv.org/abs/2401.05761>.
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [9] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wan-jia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha

参考

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov 和 Ramachandran Ramjee. 使用 Sarathi-Serve 克服 LLM 推理中的吞吐量与延迟之间的权衡。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24), 第 117-134 页, 加利福尼亚州圣克拉拉, 2024 年 7 月。USENIX 协会。ISBN 978-1-939133-40-3。URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>。
- [2] 布兰登·巴特勒、余思行、Arya Mazaheri 和 Ali Jannesari. Pipeinfer: 使用异步管道推测加速 llm 推理, 2024 年。URL <https://arxiv.org/abs/2407.11798>。
- [3] 蔡天乐, 李玉红, 耿正阳, 彭洪武, Jason D. Lee, 陈德明, Tri Dao. Medusa: 具有多个解码头的简单 llm 推理加速框架, 2024 年。URL <https://arxiv.org/abs/2401.10774>。
- [4] 凯文·马特·卡拉曼西翁。大型语言模型与搜索引擎: 评估不同信息检索场景的用户偏好, 2024 年。URL <https://arxiv.org/abs/2401.05761>。
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre 和 John Jumper. 通过推测采样加速大型语言模型解码, 2023 年。URL <https://arxiv.org/abs/2302.01318>。
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever 和 Wojciech 扎伦巴。评估在代码上训练的大型语言模型, 2021 年。URL <https://arxiv.org/abs/2107.03374>。
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever 和 Wojciech 扎伦巴。评估在代码上训练的大型语言模型, 2021 年。URL <https://arxiv.org/abs/2107.03374>。
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse 和 John Schulman. 培训验证者解决数学应用题, 2021 年。URL <https://arxiv.org/abs/2110.14168>。
- [9] DeepSeek-AI, 郭大亚, 杨德建, 张浩伟, 宋俊晓, 张若宇, 徐润鑫, 朱启豪, 马世荣, 王培毅, 毕晓, 张晓康, 余兴凯, 吴宇, 吴正峰, 苟志斌, 邵志宏, 李卓书, 高子怡, 刘爱新, 薛冰, 王冰轩, 吴博超, 冯北, 成达卢, 成刚, 邓成奇, 张晨宇, 阮冲, 戴大麦, 陈德利, 吉东杰, 李二航, 林方云, 代福聪, 罗福丽, 郝广博, 陈冠廷, 李国伟, 张汉, 鲍汉, 徐汉伟, 王浩成, 丁洪辉, 辛华建, 高华作, 曲辉, 李辉, 郭建中, 李嘉世, 王嘉伟, 陈景昌, 袁景阳, 邱俊杰, 李俊龙, 蔡俊龙, 倪佳琪, 梁健, 陈金, 董凯, 胡凯, 高凯歌, 关康, 黄可欣, 于快, 王Lean, 张乐从, 赵亮, 王立同, 张立跃, 徐雷, 夏乐一, 张明川, 张明华, 唐明辉, 李萌, 王妙君, 李明明, 田宁, 黄盼盼, 张鹏, 王前程, 陈勤宇, 杜秋实, 葛瑞琪, 张瑞松, 潘瑞哲, 王润吉, 陈荣杰, 金瑞良, 陈如意, 陆商豪, 周商彦, 陈善煌, 叶胜峰, 王世宇, 于水平, 周顺风, 潘舒婷, 李三胜, 周爽, 吴少青, 叶胜峰, 云涛, 裴天, 孙天宇, 王涛, 曾旺丁, 赵万嘉, 刘文, 梁文峰, 高文军, 于文钦, 张文涛, 肖文龙, 安伟, 刘晓东, 王晓涵, 陈晓康, 聂晓涛, 程鑫, 刘鑫, 谢鑫, 刘兴超, 杨鑫宇, 李新元, 苏学成, 林旭恒, 李XQ, 金相跃, 沉晓金, 小莎

- Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [10] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, Lansong Diao, Xiaoyong Liu, and Wei Lin. Dapple: A pipelined data parallel approach for training large models, 2020. URL <https://arxiv.org/abs/2007.01045>.
- [11] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. Longcoder: A long-range pre-trained language model for code completion, 2023. URL <https://arxiv.org/abs/2306.14893>.
- [12] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024. URL <https://arxiv.org/abs/2401.14196>.
- [13] Walid Hariri. Unlocking the potential of chatgpt: A comprehensive exploration of its applications, advantages, limitations, and future directions in natural language processing, 2025. URL <https://arxiv.org/abs/2304.02017>.
- [14] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism, 2019. URL <https://arxiv.org/abs/1811.06965>.
- [15] Maliheh Izadi, Jonathan Katzy, Tim van Dam, Marc Otten, Razvan Mihai Popescu, and Arie van Deursen. Language models for code completion: A practical evaluation, 2024. URL <https://arxiv.org/abs/2402.16197>.
- [16] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [18] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. URL <https://arxiv.org/abs/2211.17192>.
- [19] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees, 2024. URL <https://arxiv.org/abs/2406.16858>.
- [20] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024. URL <https://arxiv.org/abs/2306.00978>.
- [21] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, and Winston Hu. Parallel speculative decoding with adaptive draft length, 2024. URL <https://arxiv.org/abs/2408.11850>.
- [22] Xiaoxuan Liu, Cade Daniel, Langxiang Hu, Woosuk Kwon, Zhuohan Li, Xiangxi Mo, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Optimizing speculative decoding for serving large language models using goodput, 2024. URL <https://arxiv.org/abs/2406.14066>.
- [23] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachelgen: Kv cache compression and streaming for fast large language model serving, 2024. URL <https://arxiv.org/abs/2310.07240>.
- [24] Bradley McDanel. Amusd: Asynchronous multi-device speculative decoding for llm acceleration, 2024. URL <https://arxiv.org/abs/2410.17375>.
- [25] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang,

陈晓文、王晓翔、宋新楠、周欣怡、王先祖、单新霞、李玉坤、王玉清、魏云霞、张阳、徐艳红、李耀、赵耀、孙耀峰、王耀辉、余一凡、张一超、石一帆、熊一良、何英、朴一石、王一松、谭一轩、马一阳、刘一元、郭永强、袁欧玉端、龚跃、邹宇恒、何宇佳、熊云帆、罗宇翔、游宇翔、刘宇轩、周宇扬、朱永新、徐彦宏、黄艳平、李耀辉、郑毅、朱宇辰、马云贤、唐英、查宇坤、严雨婷、任泽辉、沙张丽、付哲、徐哲安、谢振达、张正彦、郝哲文、马志成、严志刚、吴志宇、谷子辉、朱子佳、刘子君、李子林、谢子伟、宋子阳、潘子正、黄振、徐志鹏、张忠宇和张振。Deepseek-r1: 通过强化学习激励 llms 的推理能力, 2025 年。URL <https://arxiv.org/abs/2501.12948>。

[10] 范世清, 易蓉, 陈萌, 曹宗彦, 王思宇, 郑振, 吴川, 龙国平, 杨军, 夏立雪, 刁兰松, 刘晓勇, 林伟。Dapple: 一种用于训练大型模型的管道数据并行方法, 2020 年。URL <https://arxiv.org/abs/2007.01045>。

[11] 郭大亚, 徐灿文, 段南, 尹健, 朱利安·麦考利。Long coder: 用于代码补全的远程预训练语言模型, 2023 年。URL <https://arxiv.org/abs/2306.14893>。

[12] 郭大亚, 朱启浩, 杨德建, 谢振达, 董凯, 张文涛, 陈冠廷, 毕晓, 吴勇, 李玉坤, 罗富丽, 熊英飞, 梁文峰。Deepseek-coder: 当大型语言模型遇到编程时——代码智能的兴起, 2024 年。URL <https://arxiv.org/abs/2401.14196>。

[13] 瓦利德·哈里里。释放 chat-gpt 的潜力: 全面探索其在自然语言处理中的应用、优势、局限性和未来方向, 2025 年。URL <https://arxiv.org/abs/2304.02017>。

[14] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, 和 Zhifeng Chen。Gpipe: 使用管道并行性对巨型神经网络进行高效训练, 2019。URL <https://arxiv.org/abs/1811.06965>。

[15] Maliheh Izadi, Jonathan Katzy, Tim van Dam, Marc Otten, Razvan Mihai Popescu 和 Arie van Deursen。代码补全的语言模型: 实践评估, 2024 年。URL <https://arxiv.org/abs/2402.16197>。

[16] 汤姆·科维亚特科斯基、珍妮玛丽亚·帕洛玛基、奥利维亚

Redfield、Michael Collins、Ankur Parikh、Chris Alberti、Danielle Epstein、Illia Polosukhin、Jacob Devlin、Kenton Lee、Kristina Toutanova、Llion Jones、Matthew Kelcey、Ming-Wei Chang、Andrew M. Dai、Jakob Uszkoreit、Quoc Le 和 Slav Petrov。自然问题: 问答研究的基准。计算语言学协会汇刊, 7:452–466, 2019。doi: 10.1162/tacl_a_00276。网址 <https://aclanthology.org/Q19-1026/>。[17] Woosuk Kwon, Zhuohan Li, 庄思源, Yingheng, Lianmin Cheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica。使用 pagedattention 服务的大型语言模型的高效内存管理, 2023。URL <https://arxiv.org/abs/2309.06180>。[18] Yaniv Leviahtan, Matan Kalman 和 Yossi Matias。通过推测性解码从 Transformer 进行快速推理, 2023 年。URL <https://arxiv.org/abs/2211.17192>。[19] 李玉辉, 魏方云, 张超, 张红阳。Eagle-2: 使用动态草稿树更快地推理语言模型, 2024 年。URL <https://arxiv.org/abs/2406.16858>。[20] 吉林, 唐家明, 唐浩天, 尚阳, 陈伟明, 王伟臣, 肖光轩, 党兴宇, 甘闯, 韩松。Awq: llm 压缩和加速的激活感知权重量化, 2024 年。URL <https://arxiv.org/abs/2306.00978>。[21] 刘天宇, 李云, 吕启坦, 刘凯, 朱建辰, 胡文斯顿。具有自适应草稿长度的并行推测解码, 2024 年。URL <https://arxiv.org/abs/2408.11850>。[22] 刘晓轩, Cade Daniel, 胡郎翔, Woosuk Kwon, 李卓瀚, 莫湘西, Alvin Cheung, 邓志杰, Ion Stoica 和张浩。使用 goodput 优化推测解码以服务大型语言模型, 2024。URL <https://arxiv.org/abs/2406.14066>。[23] 刘玉涵, 李汉辰, 程一华, Siddhant Ray, 黄宇阳, 张启正, 杜坤泰, 姚嘉怡, 卢珊, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, Junchen Jiang。Cachegen: 用于快速大型语言模型服务的 Kv 缓存压缩和流式传输, 2024 年。URL <https://arxiv.org/abs/2310.07240>。[24] 布拉德利·麦克丹尼尔。Amusd: 用于 llm 加速的异步多设备推测解码, 2024 年。URL <https://arxiv.org/abs/2410.17375>。[25] 苗旭鹏, Gabriele Oliaro, 张志豪, 程新浩, 王泽宇, 张正新, Rae Ying Yee Wong, Alan Zhu, Lijie Yang,

- Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, ASPLOS '24. ACM, April 2024. doi: 10.1145/3620666.3651335. URL <http://dx.doi.org/10.1145/3620666.3651335>.
- [26] Ramesh Nallapati, Bowen Zhou, Cicero Nogueira dos santos, Caglar Gulcehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence rnns and beyond, 2016. URL <https://arxiv.org/abs/1602.06023>.
- [27] Nvidia. Nvidia/tensorrt-llm: A tensorrt toolbox for optimized large language model inference. <https://github.com/NVIDIA/TensorRT-LLM>.
- [28] OpenAI. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- [29] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting, 2024. URL <https://arxiv.org/abs/2311.18677>.
- [30] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL <https://arxiv.org/abs/1909.08053>.
- [31] Tatsu-Lab. Tatsu-lab/stanford-alpaca: Code and documentation to train stanford’s alpaca models, and generate the data. https://github.com/tatsu-lab/stanford_alpaca.
- [32] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023. URL <https://github.com/NVIDIA/cutlass>.
- [33] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- [36] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models, 2024. URL <https://arxiv.org/abs/2211.10438>.
- [37] Haoyi Xiong, Jiang Bian, Yuchen Li, Xuhong Li, Mengnan Du, Shuaiqiang Wang, Dawei Yin, and Sumi Helal. When search engine services meet large language models: Visions and challenges, 2024. URL <https://arxiv.org/abs/2407.00128>.
- [38] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024. URL <https://arxiv.org/abs/2407.10671>.
- [39] Fan Yang, Zheng Chen, Ziyang Jiang, Eunah Cho, Xiaojiang Huang, and Yanbin Lu. Palr: Personalization aware llms for recommendation, 2023. URL <https://arxiv.org/abs/2305.07622>.
- [40] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-28-1. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [41] Jiatao Zhang, Lanling Tang, Yufan Song, Qiwei Meng, Haofu Qian, Jun Shao, Wei Song, Shiqiang Zhu, and Jason Gu. Fltrnn: Faithful long-horizon task planning for robotics with large language models. In *2024 IEEE International Conference on Robotics*

施潇湘、施楚南、陈卓明、Daiyaan Arfeen、Reyna Abhyankar 和贾志浩。Specinfer: 通过基于树的推测推理和验证来加速大型语言模型。第 29 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 3 卷, ASPLOS ' 24. ACM, 2024 年 4 月。doi: 10.1145/3620666.3651335。网址 <https://dx.doi.org/10.1145/3620666.3651335>。[26] Ramesh Nalapati, 周博文, Cicero Nogueira dos Santos, Caglar Gulcehre, Bing Xing。使用序列到序列 rnn 进行抽象文本摘要, 2016 年。URL <https://arxiv.org/abs/1602.06023>。[27] 英伟达。Nvidia/tensorrt-llm: 用于优化大语言模型推理的 tensorrt 工具箱。 <https://github.com/NVIDIA/TensorRT-LLM>。[28] 开放人工智能。Gpt-4 技术报告, 2024 年。URL <https://arxiv.org/abs/2303.08774>。[29] Pratyush Patel, Esha Chouksey、Chaojie Zhang、Aashaka Shah、Íñigo Goiri、Saeed Maleki 和 Ricardo Bianchini。Splitwise: 使用相分裂的高效生成 llm 推理, 2024 年。URL <https://arxiv.org/abs/2311.18677>。[30] Mohammad Shoeybi、Mostofa Patwary、Raul Puri、Patrick LeGresley、Jared Casper 和 Bryan Catanzaro。Megatron-lm: 使用模型并行性训练数十亿参数语言模型, 2020。URL <https://arxiv.org/abs/1909.08053>。[31] 达实验室。Tatsu-lab/stanford-alpaca: 用于训练斯坦福羊驼模型并生成数据的代码和文档。 https://github.com/tatsu-lab/stanford_alpaca。[32] Vijay Thakkar、Pradeep Ramani、Cris Cecka、Aniket Shivam、Lu Honghao、Ethan Yan、Jack Kosaian、Mark Hoemmen、吴海成、Andrew Kerr、Matt Nicely、Duane Merrill、Dustin Blasig、Fengqi Qiao、Piotr Majcher、Paul Springer、Markus Hohnerbach、Jin Wang 和 Manish Gupta。CUTLASS, 2023 年 1 月。URL <https://github.com/NVIDIA/cutlass>。[33] Hugo Touvron、Thibaut Lavril、Gautier Izacard、Xavier Martinet、Marie-Anne Lachaux、Timothée Lacroix、Baptiste Rozière、Naman Goyal、Eric Hambro、Faisal Azhar、Aurelien Rodriguez、Armand Joulin、Edouard Grave 和 Guillaume Lample。Llama: 开放高效的基础语言模型, 2023 年。URL <https://arxiv.org/abs/2302.13971>。[34] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N. Gomez、Lukasz Kaiser 和 Illia Polosukhin。注意力全是你

需要, 2023 年。URL <https://arxiv.org/abs/1706.03762>。[35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou。思维链提示在大型语言模型中引发推理, 2023 年。URL <https://arxiv.org/abs/2201.11903>。[36] 肖光轩, 林吉, Mickael Seznec, 吴浩, Julien Demouth, 宋瀚。Smoothquant: 大型语言模型的准确高效的训练后量化, 2024 年。URL <https://arxiv.org/abs/2211.10438>。[37] 熊浩一, 卞江, 李雨辰, 李旭红, 杜蒙南, 王帅强, 尹大伟, 苏米·赫拉尔。当搜索引擎服务遇到大型语言模型: 愿景与挑战, 2024 年。URL <https://arxiv.org/abs/2407.00128>。[38] 安阳、杨宝松、惠斌源、郑波、博文于、常周、李成鹏、李成远、刘大一恒、黄飞、董冠廷、魏浩然、林焕、唐家龙、王家林、杨建、涂建红、张建伟、马建新、杨建新、徐金、周景仁、白金泽、何金正、林俊阳、党凯、陆克明、克勤陈、可欣、李美、薛明峰、娜妮、张培、王鹏、彭茹、门睿、高瑞泽、林润吉、王世杰、白帅、谭思南、朱天航、李天浩、刘天宇、葛文斌、邓晓东、周小欢、任兴章、张新宇、魏西平、任绪安成、刘学静、杨帆、姚洋、张宜昌、万宇、褚云飞、刘玉琼、崔泽宇、张真如、郭志芳和范志浩。Qwen2 技术报告, 2024 年。URL <https://arxiv.org/abs/2407.10671>。[39] 杨帆, 陈铮, 蒋紫妍, 曹尤娜, 黄晓江, 陆彦斌。Palr: 个性化感知 llms 推荐, 2023 年。URL <https://arxiv.org/abs/2305.07622>。[40] Gyeong-In Yu、Joo Seong Jeong、Geon-Woo Kim、Soojeong Kim 和 Byung-Gon Chun。Orca: 基于 Transformer 的生成模型的分布式服务系统。第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22), 第 521–538 页, 加利福尼亚州卡尔斯巴德, 2022 年 7 月。USENIX 协会。ISBN 978-1-939133-28-1。网址 <https://www.usenix.org/conference/osdi22/presentation/yy>。[41] 张家涛, 唐兰陵, 宋宇凡, 孟其伟, 钱浩夫, 邵军, 宋伟, 朱世强, 顾杰森。Fltrnn: 具有大型语言模型的机器人的忠实长期任务规划。2024 年 IEEE 国际机器人会议

- and Automation (ICRA), pages 6680–6686, 2024. doi: 10.1109/ICRA57147.2024.10611663.
- [42] Siyan Zhao, Daniel Israel, Guy Van den Broeck, and Aditya Grover. Prepacking: A simple method for fast prefilling and increased throughput in large language models, 2024. URL <https://arxiv.org/abs/2404.09529>.
 - [43] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.
 - [44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.
 - [45] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pages 193–210, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
 - [46] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection, 2025. URL <https://arxiv.org/abs/2412.04455>.

和自动化 (ICRA), 第 6680–6686 页, 2024 年。doi: 10.1109/ICRA57147.2024.10611663。

[42] 赵思燕、丹尼尔·伊斯雷尔、盖伊·范·登·布罗克和阿迪亚·格罗弗。预打包：一种在大型语言模型中快速预填充和增加吞吐量的简单方法，2024 年。URL <https://arxiv.org/abs/2404.09529>. [43] 郑联民、蒋伟林、盛英、庄思源、吴章浩、庄永浩、林子、李卓涵、李大成、Eric P. Xing、张浩、Joseph E. Gonzalez 和 Ion Stoica。使用 mt-bench 和聊天机器人竞技场对 llm-as-a-judge 进行评审，2023 年。URL <https://arxiv.org/abs/2306.05685>. [44] 郑连民、尹良胜、谢志强、孙初跃、黄杰夫、于浩、曹诗一、Christos Kozyrakis、Ion Stoica、Joseph E. Gonzalez、Clark Barrett、盛英。Sglang：结构化语言模型程序的高效执行，2024 年。URL <https://arxiv.org/abs/2312.07104>. [45] 钟银民，刘胜宇，陈俊达，胡建波，朱一波，刘焯哲，金鑫，张浩。DistServe：分解预填充和解码，以实现良好的优化大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)，第 193-210 页，加利福尼亚州圣克拉拉，2024 年 7 月。USENIX 协会。ISBN 978-1-939133-40-3。网址 <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>。 [46] 周恩深，苏奇，迟成，张志正，王中原，黄铁军，卢胜，王鹤。代码即监视器：用于被动式和主动式机器人故障检测的约束感知视觉编程，2025 年。URL <https://arxiv.org/abs/2412.04455>。
